: Training Efficient Repository Explorer for Coding Agents

Shaoqiu Zhang · Maoquan Wang · Yuling Shi · Yuhang Wang · Xiaodong Gu · Yongqiang Yao · Tori Gong · Sheng Chen · Rao Fu · Anisha Agarwal · Spandan Grag · Gabriel Ryan · Colin Merkel · Yufan Huang · Shengyu Fu

ABSTRACT

Large Language Model (LLM) coding agents have achieved strong results on software engineering tasks, yet repository exploration remains a major bottleneck: locating relevant code consumes substantial token budget and pollutes the agent's context with irrelevant snippets. In most agents, the same model explores the repository and solves the task, leaving exploratory reads and searches in the solver's history. We present , a dedicated exploration subagent that separates repository exploration from solving. Invoked on demand, issues parallel tool calls and returns concise file paths and line ranges as focused context. is powered by specialized exploration models spanning 4B–30B parameters. We bootstrap them from strong reference-model trajectories and refine them with task-grounded rewards for broad first-turn search, multi-turn evidence gathering, and precise citation generation. Across SWE-bench Multilingual, SWE-bench Pro, and SWE-QA, integrating into Mini-SWE-Agent improves end-to-end resolution rates up to ***5.5%*** *while reducing coding-agent token consumption up to* ***60%****, with marginal overhead. These results show that repository exploration can be separated from solving and handled effectively by specialized models. Code and data:* ***https://github.com/microsoft/fastcontext***

[a]Equal contribution.
[b]Work done during a project at Microsoft CoreAI.
[c]Corresponding author.

1 Introduction

Coding agents have become a prominent approach for automated software engineering, capable of handling everything from localized code refactoring to repository-level question answering [29, 32, 35, 41]. Industry-leading assistants such as Claude Code, Codex, GitHub Copilot CLI, and Cursor [1, 6, 11, 17] continuously push these boundaries through increasingly sophisticated mechanisms, with the recent rise of specialized subagents marking a vital frontier in the field. However, the repository exploration mechanisms powering these advanced systems are typically

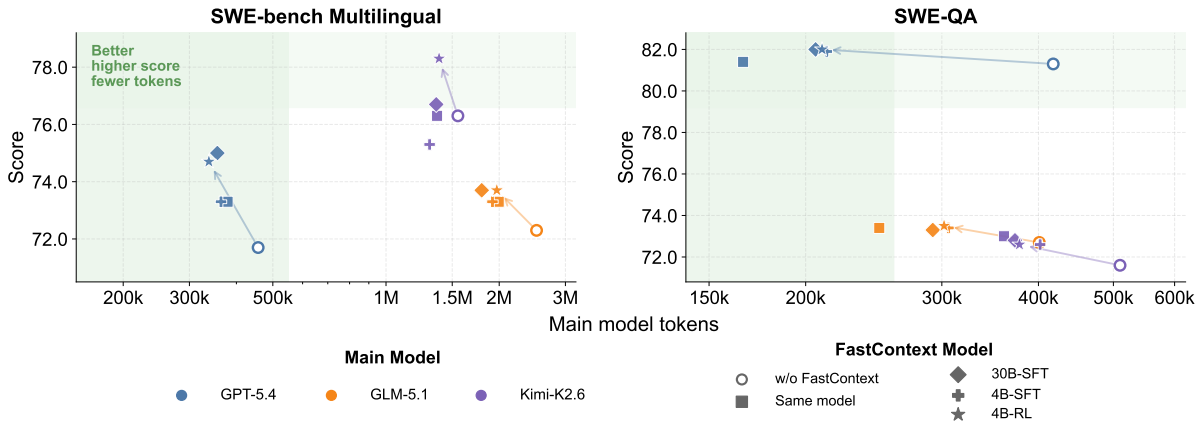

FIGURE 1 SWE-bench Multilingual and SWE-QA score versus main-model token usage, where shifts coding agents toward a better score–token tradeoff: cost less and archive more.

proprietary, leaving the research community without open training and evaluation recipes to build upon.

Benchmarks such as SWE-bench and SWE-QA exemplify the community's shift from isolated coding problems toward realistic tasks that require navigating large, multi-file codebases [13, 20]; newer suites further broaden this setting across harder, multilingual, and decontaminated evaluations [3, 9]. Across these tasks, an agent must explore the repository to identify relevant files and code regions before it can reason about a fix or answer. As such, this exploration stage plays a critical role in both task success and inference efficiency. We use *main agent* to refer to the coding agent responsible for solving the task, and *subagent* to refer to a specialized helper that the main agent can invoke for a narrower repository-exploration job.

However, repository exploration remains a costly part of current agent trajectories, as also observed by SWE-Pruner for coding-agent context consumption [30]. Our preliminary analysis shows that reading and searching account for a large share of tool-use turns and main-agent total tokens in our trajectories. This cost is consistent with prior systems that devote substantial machinery to localization, search, or code-search training before patch generation [27, 32, 41]. When exploration misses key files or accumulates irrelevant snippets, the main agent must reason from noisy context and may spend later turns repairing a mistaken hypothesis rather than addressing the true cause.

Recent work has begun to study repository context selection. Graph- and structure-guided methods use program information to improve localization [5, 12, 28, 41]; retrieval, compression, and workflow methods select or condense repository context before generation [23, 24, 32, 39]; and RL or search-based systems train or refine repository-search agents [18, 27]. These studies show that better context can improve software agents, but they leave open the role of a lightweight, trained explorer that can coexist with a standard main agent. Many localization-oriented evaluations emphasize file- or function-level metrics, while end-to-end gains are often reported inside specialized pipelines; in either case, broad localization can still hand the main agent noisy or overly broad context that does not fit its actual decision point. Moreover, graph construction, specialized workflows, or frontier-model exploration can be too expensive to serve as a simple

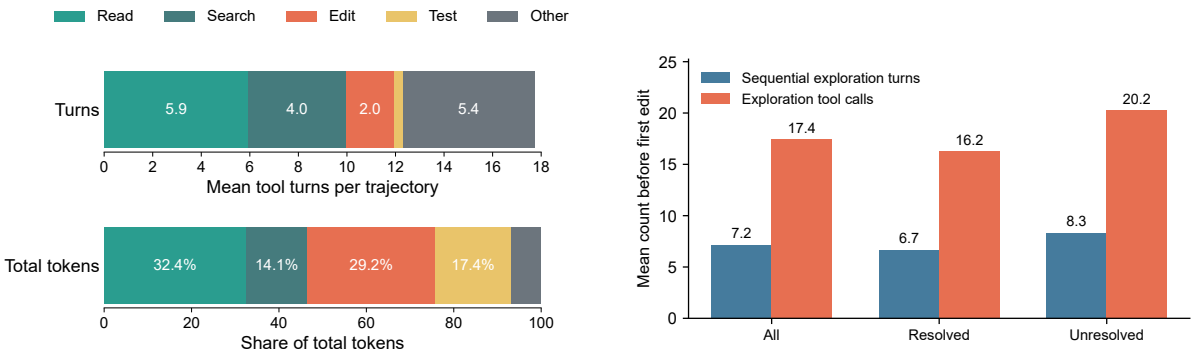

FIGURE 2 Trajectory analysis of GPT-5.4-high with Mini-SWE-Agent. Left: reading and searching dominate both tool-use turns and frontier-model prompt-token usage across the full trajectory. Right: before the first edit, agents execute many exploration tool calls across multiple sequential exploration turns, motivating an exploration component that can issue parallel tool calls outside the main solver trajectory.

reusable component for minimal frameworks such as Mini-SWE-Agent.

To address these limitations, we propose , a dedicated exploration *subagent* that isolates repository search from the main solving agent. receives a natural-language issue description or a request for repository context, iteratively plans and executes *parallel* tool calls for file reading, glob matching, and regex search, and returns concise file paths with line ranges as focused context for the main agent. By moving broad exploration into a separate subagent, the main agent can receive cleaner repository evidence rather than carrying the irrelevant code accumulated during navigation.

To make this delegation inexpensive, we train specialized exploration models spanning 4B–30B parameters with supervised fine-tuning and reinforcement learning, bootstrapping from strong reference-model trajectories and refining the models with task-grounded rewards. Integrated into Mini-SWE-Agent, improves end-to-end resolution rates up to **5.5%** while reducing main model token consumption up to **60%** on SWE-bench Multilingual, SWE-bench Pro, and SWE-QA benchmarks.

In summary, our contributions are as follows:

- We propose , an exploration subagent that decouples repository search from main-agent solving, which performs iterative, parallel tool use and returns grounded file-and-line citations as compact context.
- We train a family of specialized exploration models (4B–30B) via SFT and RL, teaching them to start with broad search, gather evidence over multiple turns, and produce precise final file-line evidence.
- End-to-end experiments with Mini-SWE-Agent on 3 benchmarks show that our exploration subagents improve resolution accuracy up to **5.5%** while reducing main model token consumption up to **60%**.

2 Preliminary Analysis

To understand whether repository exploration is a distinct bottleneck worth delegating, we analyze all 300 GPT-5.4-high trajectories produced by Mini-SWE-Agent on the full SWE-bench Multilingual set to quantify where the main agent spends its budget. This analysis follows recent observations from SWE-Pruner that coding agents can spend substantial token budget on read/search context and therefore benefit from context pruning [30]. Figure 2 categorizes tool calls into reading, searching, editing, testing, and other actions, and reports their shares in both tool-use turns and total tokens. For assistant turns containing multiple tool calls, we distribute the turn and token counts across the called tools. Reading and searching dominate the full trajectory: together they account for 9.96 of 17.72 tool-use turns per instance on average, or 56.2% of all tool-use turns, and consume 46.5% of the main agent's total tokens. This token share directly corresponds to inference cost and to context that the frontier agent must carry forward.

The turn-level view reveals a second bottleneck: exploration is also a long sequential prelude before editing. Among the 284 trajectories where a first source-code edit can be identified, the agent starts editing at turn 8.47 on average. Even with parallel tool calls enabled, the median trajectory still requires six sequential exploration turns and 15.5 exploration tool calls before the first edit. Unresolved trajectories are associated with more pre-edit exploration turns than resolved ones, with 8.34 versus 6.67 turns on average. This pattern is expected: harder issues naturally require more search. Rather than proving that exploration itself causes failure, this comparison exposes where a separate exploration component can help. When hard instances demand many exploratory turns, moving this search outside the main agent creates an opportunity to reduce the context and token burden that the solver must carry. At the same time, hard instances often require sharper guidance toward the few files and lines that determine the fix; a trained explorer can potentially provide this guidance by returning compact evidence instead of a long trail of exploratory context. These results motivate as a reusable exploration component: repository exploration is expensive enough to affect cost, yet structured enough to be delegated to a subagent that performs parallel search and returns compact file-and-line context for the main agent.

3 Method

We present , a lightweight exploration subagent that separates repository search from main-agent issue solving. Given an issue or a request for repository context, explores the target repository with read-only tools and returns compact file-line evidence for the main agent to consume. The main text focuses on the delegation contract and training recipe; implementation details for prompts, runtime limits, and reward thresholds are deferred to the appendix. The method consists of a simple runtime harness and two training stages: supervised fine-tuning (SFT) to initialize exploration behavior, followed by reinforcement learning (RL) to align the returned evidence with task-relevant code locations. Figure 3 gives an overview of this workflow: the explorer gathers repository evidence through parallel tools, compresses it into file-line citations, and passes the resulting context to the main agent.

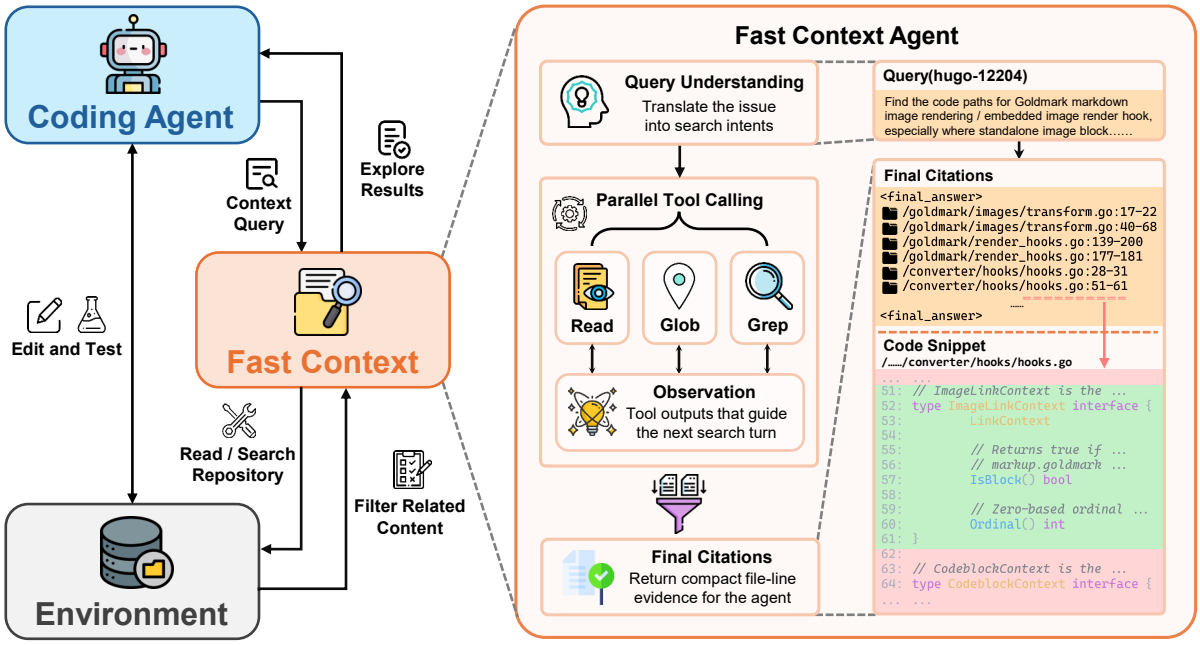

FIGURE 3 Overview of . Left: in the end-to-end agent loop architecture, where the coding agent delegates repository exploration to and receives compact file-line evidence before continuing with editing and testing. Right: the internal architecture and an example, showing query understanding, parallel tool calling, observations, and final citations with supporting code snippets.

3.1 Subagent Architecture

is a runtime delegation mechanism: the main agent delegates repository exploration to the explorer, which returns evidence rather than a patch. The main agent then consumes this narrowed context, avoiding the long sequence of exploratory reads and searches that would otherwise remain in its own conversation history.

The subagent deliberately exposes only three language-agnostic tools: READ for line-numbered file contents, GLOB for path discovery, and GREP[1] for regex search over repository text. At each turn, the explorer either issues one or more tool calls or stops with a final evidence list. Multiple tool calls in the same turn are executed in parallel, allowing the explorer to cover complementary hypotheses before synthesizing the observations.

The output contract is a compact final answer block containing file paths and line ranges, optionally followed by short relevance notes. A typical answer is:

```
<final_answer>
/src/router.py:42-58 (Router definition)
/tests/test_router.py:101-119
</final_answer>
```

This format makes the explorer's output directly consumable as focused context for the main agent.

[1] https://github.com/BurntSushi/ripgrep

3.2 Policy Initialization with Supervised Fine-Tuning

We train the initial exploration policy through supervised fine-tuning. Specifically, we construct training examples from SWE-bench-style repository-exploration tasks, each containing an issue description, a repository snapshot, and workspace metadata. Instead of training only on final locations, we decompose supervision according to the runtime behaviors the subagent must perform. This exposes the model to the full exploration loop, from broad initial search to observation-driven refinement and compact citation output.

We construct 2,954 filtered SFT examples from Sonnet 4.6 exploration traces, split into three sources that match the runtime behavior of the subagent.

The first source, `parallel_toolcalls`, targets *broad first-turn search*: the reference model is prompted with the query and top-level directory listing and asked to issue nonredundant parallel tool calls that cover complementary signals such as path patterns, symbols, and likely entry points. The second source, `multiturn_traj`, targets *multi-turn evidence gathering*: we retain reference-model trajectories, including system and user messages, assistant tool-call arguments, and raw tool observations. The third source, `linerange`, targets *precise citation generation*: we provide retrieved file contents and ask the reference model to emit only a narrow `<final_answer>` block with relevant file-and-line ranges. Additional construction details are deferred to Appendix A.

These sources are merged into a multi-turn chat dataset $\mathcal{D}_{\text{sft}}$ with the same tool schemas used at inference time. Let x denote the visible conversation prefix and y denote the reference assistant continuation, which may be natural-language text, tool-call arguments, or a final citation block. We optimize an assistant-token-only objective

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{D}_{\text{sft}}|} \sum_{(x,y)\in\mathcal{D}_{\text{sft}}} \sum_{t=1}^{|y|} m_t \log p_\theta(y_t \mid x, y_{<t}), \tag{1}$$

where m_t masks out non-assistant tokens and keeps both ordinary assistant text and structured tool call arguments in the loss. We fine-tune explorer checkpoints with this objective; model scales and optimization details are reported in Section 4.1 and Appendices A–A.2.

3.3 Policy Refinement with Reinforcement Learning

SFT imitation does not directly optimize whether the final citations cover the code locations needed to solve the issue. We therefore refine the explorer with task-grounded RL. We construct a 400-prompt RL set from issue-resolution tasks with reference patches. Each prompt contains the explorer instruction, workspace metadata, a top-level directory listing, and a natural-language repository exploration query. For each training instance, we parse the reference patch into target file-and-line ranges and use them as exploration labels, replacing teacher continuations with a patch-derived `<final_answer>` target. Appendix A.3 gives the data construction and rollout details. The model is rolled out as the actual subagent: it receives the same repository workspace and instruction prompt, interacts with Read, Glob, and Grep for a bounded number of turns, and finally produces a `<final_answer>` block.

The reward is deterministic and tied to the explorer's output contract. It combines patch-

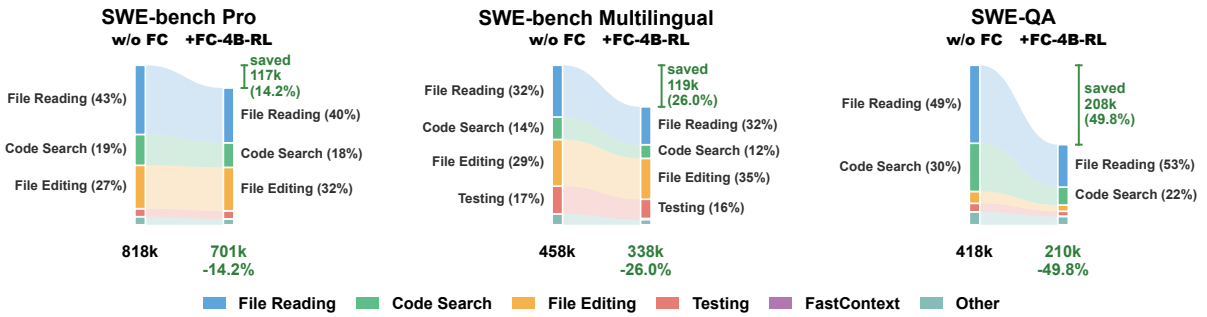

FIGURE 4 Breakdown of GPT-5.4 main-agent total tokens before and after adding with the FC-4B-RL explorer. Each panel compares direct solving against FastContext-augmented solving on one benchmark, with tokens grouped by action category. substantially reduces main-agent context consumption, especially from file reading and code search, while adding only a small FastContext invocation overhead.

derived localization accuracy, structured parallel exploration, and output-validity penalties. Let G_f and G_l denote the target file and line sets induced by the reference patch, and let P_f and P_l denote the corresponding sets parsed from the model's final citations. The scalar reward is

$$R = \underbrace{F_1(P_f, G_f) + F_1(P_l, G_l)}_{\text{task outcome}} + \underbrace{r_{\text{parallel}}}_{\substack{\text{parallel} \\ \text{function call}}} - \underbrace{r_{\text{format}}}_{\text{penalty}}. \tag{2}$$

Here the task-outcome term is the sum of file-level and line-level F1 after path normalization, with zero score for empty sets; r_{parallel} gives a small bonus to bounded multi-call exploration; and r_{format} rejects empty, overly long, malformed, or excessive-fan-out outputs. Exact thresholds and rollout settings are deferred to Appendix A.4. We initialize from the SFT checkpoint and optimize with GRPO [22], sampling multiple trajectories per prompt. This stage aligns the model with the practical goal of returning a small citation set that gives the main agent the code regions most likely to matter.

4 Experiments

We evaluate from two complementary perspectives. First, we measure end-to-end task performance when FastContext is attached to a coding agent, where the main question is whether focused repository evidence improves success while reducing the main agent's context cost. Second, we evaluate standalone exploration quality, isolating whether the explorer can recover the code locations implicated by a reference patch.

4.1 Experimental Setup

End-to-end benchmarks. We use three benchmarks for full agent evaluation. SWE-bench Multilingual contains 300 issue-resolution instances spanning multiple programming languages. SWE-bench Pro [9] provides more challenging software-engineering tasks; we evaluate on a fixed randomly sampled 200-instance subset listed in Appendix J. SWE-QA [20] evaluates repository-level question answering, where the agent must locate and reason over relevant code rather than

		SWE-bench Multilingual			SWE-bench Pro			SWE-QA		
Main Agent	**Subagent**	**Score**	**Tokens**	**Turns**	**Score**	**Tokens**	**Turns**	**Score**	**Tokens**	**Turns**
GPT-5.4	*w/o Explore*	71.7	457k	17.7	46.0	818k	20.7	81.3	418k	15.7
	GPT-5.4	73.3 ↑1.6	379k ↓17.1%	18.3	**51.5** ↑5.5	703k ↓14.1%	23.7	81.4 ↑0.1	**166k** ↓60.3%	9.8
	FC-30B-SFT	**75.0** ↑3.3	356k ↓22.1%	18.2	49.0 ↑3.0	**688k** ↓15.9%	23.5	**82.0** ↑0.7	206k ↓50.7%	11.2
	FC-4B-SFT	73.3 ↑1.6	364k ↓20.4%	18.3	47.0 ↑1.0	689k ↓15.8%	23.2	81.9 ↑0.6	213k ↓49.0%	11.6
	FC-4B-RL	74.7 ↑3.0	**338k** ↓26.0%	18.3	48.5 ↑2.5	701k ↓14.3%	23.5	**82.0** ↑0.7	210k ↓49.8%	11.4
GLM-5.1	*w/o Explore*	72.3	2514k	73.9	17.5	2692k	67.4	72.7	401k	27.7
	GLM-5.1	73.3 ↑1.0	1994k ↓20.7%	55.9	18.0 ↑0.5	2356k ↓12.5%	63.9	73.4 ↑0.7	**249k** ↓37.9%	20.4
	FC-30B-SFT	**73.7** ↑1.4	**1797k** ↓28.5%	55.0	20.0 ↑2.5	2370k ↓12.0%	64.2	73.3 ↑0.6	292k ↓27.2%	23.0
	FC-4B-SFT	73.3 ↑1.0	1919k ↓23.7%	56.9	18.0 ↑0.5	2279k ↓15.3%	64.0	73.4 ↑0.7	306k ↓23.7%	23.8
	FC-4B-RL	**73.7** ↑1.4	1971k ↓21.6%	56.6	**22.5** ↑5.0	**2210k** ↓17.9%	64.3	**73.5** ↑0.8	302k ↓24.7%	23.2
Kimi-K2.6	*w/o Explore*	76.3	1553k	55.7	31.0	2383k	68.0	71.6	510k	32.5
	Kimi-K2.6	76.3 ↑0.0	1367k ↓12.0%	50.5	32.0 ↑1.0	**2060k** ↓13.6%	58.0	**73.0** ↑1.4	**361k** ↓29.2%	24.4
	FC-30B-SFT	76.7 ↑0.4	1360k ↓12.4%	49.9	33.0 ↑2.0	2150k ↓9.8%	58.8	72.8 ↑1.2	373k ↓26.9%	26.4
	FC-4B-SFT	75.3 ↓1.0	**1306k** ↓15.9%	49.3	32.5 ↑1.5	2159k ↓9.4%	61.6	72.6 ↑1.0	402k ↓21.2%	28.0
	FC-4B-RL	**78.3** ↑2.0	1384k ↓10.9%	52.1	**33.5** ↑2.5	2158k ↓9.4%	61.1	72.6 ↑1.0	378k ↓25.9%	27.5

TABLE 1 End-to-end performance and efficiency across three benchmarks. Score denotes the benchmark success metric; Tokens and Turns are measured on the main-agent trajectory. Score deltas and token reductions are computed relative to *w/o Explore* for the same main agent. The best and the second best results are marked in **bold** and underlined.

produce a patch. For SWE-QA, we use GPT-5.4 as the LLM-as-judge evaluator. Together, these benchmarks cover multilingual and harder issue resolution, and codebase question answering.

End-to-end protocol. All end-to-end experiments use Mini-SWE-Agent as the terminal-only main-agent scaffold with GPT-5.4, GLM-5.1, and Kimi-K2.6 as main agents. For each main agent, we compare direct solving, same-model exploration where the frontier model also performs delegated search, and trained explorers, including 30B-SFT, 4B-SFT, and 4B-RL variants. The 4B models are our deployment targets because exploration should be cheap enough to run as a practical helper; 30B-SFT serves as a scaling reference, and 4B-RL tests whether task-grounded optimization can make a compact explorer competitive without a 30B RL run. In augmented settings, the main agent invokes the explorer as needed and receives compact file-line citations rather than the full exploratory trajectory. We report benchmark success, main-agent tokens, and main-agent turns, with token reductions computed against the direct-solving baseline for the same main agent. Appendix B gives runtime and overhead details.

Standalone exploration protocol and benchmark. To evaluate the explorer itself, we use a standalone patch-localization benchmark on SWE-bench Verified [13]. For each instance, we use patch-derived reference locations [27] and compute instance-wise precision, recall, and F1 at file, module, and function granularity. Predicted file-line citations are mapped into the same target space before scoring, so all methods are compared under a shared localization protocol. Baseline scaffolds, model backbones, patch parsing, line-to-symbol mapping, and edge-case handling are detailed in Appendix D.

4.2 End-to-End Results

improves end-to-end accuracy. Table 1 shows that, for every main agent and benchmark, the best explorer-augmented run outperforms direct solving. The largest gains appear on SWE-bench Pro: GPT-5.4 improves from 46.0 to 51.5, GLM-5.1 from 17.5 to 22.5, and Kimi-K2.6 from 31.0 to 33.5. SWE-bench Multilingual also improves for all three main agents, while SWE-QA shows smaller but still positive gains.

consistently saves main-agent tokens. Every explorer-augmented setting uses fewer main-agent tokens than direct solving. The largest savings occur on SWE-QA, reaching 60.3% for GPT-5.4 and 37.9% for GLM-5.1; trained explorers still save about 50% and 24–27% respectively on the same benchmark. Figure 5 further shows the per-instance total-token distributions for GPT-5.4 on SWE-bench Multilingual. All explorer variants shift the distribution toward lower token usage, indicating that the average savings in Table 1 reflect a broad reduction across instances.

The gains depend on the main agent and benchmark.GLM-5.1 has the most verbose direct trajectories, so removes hundreds of thousands of tokens on SWE-bench Multilingual and SWE-bench Pro. Kimi-K2.6 starts from a stronger and shorter baseline, so its token savings are smaller on issue-resolution tasks, but it still gains accuracy on both SWE-bench Multilingual and SWE-bench Pro. Detailed case studies are provided in Appendix C.

4.3 Ablation Analysis

Same-model exploration is not usually the best trade-off. Trained models often dominate same-model exploration in both score and tokens. For GPT-5.4 on SWE-bench Multilingual, same-model exploration reaches 73.3 with 379k tokens, while 30B-SFT reaches 75.0 with 356k tokens and 4B-RL reaches 74.7 with 338k tokens. For GLM-5.1 on SWE-bench Pro, 4B-RL improves the same-model explorer from 18.0 to 22.5 while reducing tokens from 2356k to 2210k.

The 4B-RL explorer can outperform the larger 30B-SFT explorer. On GLM-5.1 SWE-bench Pro, 4B-RL reaches 22.5 versus 20.0 for 30B-SFT while using fewer tokens. It also exceeds 30B-SFT on Kimi-K2.6 SWE-bench Multilingual and SWE-bench Pro.

RL consistently improves the compact explorer. Compared with 4B-SFT, 4B-RL improves or ties the score in all nine end-to-end settings, with clear gains on GPT-5.4 SWE-bench Pro, GLM-5.1 SWE-bench Pro, and both Kimi-K2.6 issue-resolution benchmarks. Thus, 30B-SFT serves as a larger-model reference, while 4B-RL supports our main design choice: task-grounded RL is an effective path toward small exploration subagents without requiring an expensive 30B-RL variant.

4.4 Standalone Exploration Quality

recovers patch-relevant locations more accurately. Table 2 evaluates standalone exploration using locations derived from the final patch as reference locations. Under this proxy, variants form the strongest group at file and module granularity, indicating that the end-to-end

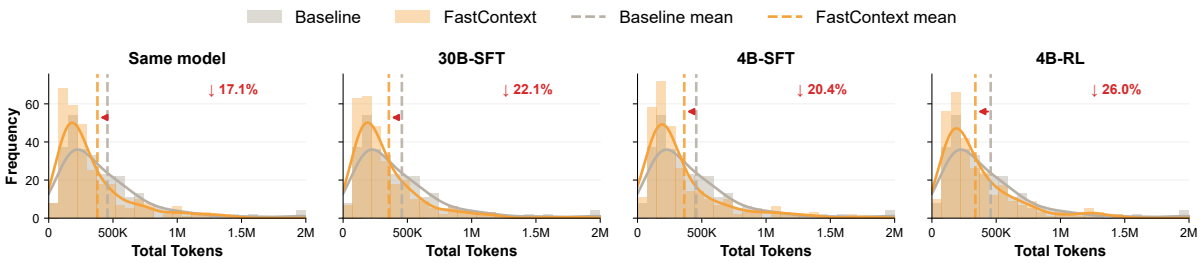

FIGURE 5 Per-instance GPT-5.4 main-agent total-token distributions on SWE-bench Multilingual. Each panel compares direct solving against one FastContext-augmented setting.

gains partly reflect better recovery of patch-relevant evidence. Frontier-model explorers serve as reference upper bounds, while the highlighted comparison follows the small-model deployment setting. Among non-frontier rows, trained checkpoints reach 73.71 file-level F1 and 60.35 module-level F1, compared with 68.57 and 50.88 for the best non- rows. The advantage is clearest at module and function level, suggesting that narrows evidence toward the code regions most likely to matter.

Training improves compact explorers. Within the scaffold, SFT substantially improves the 4B explorer at coarser granularities, raising file-level F1 from 62.57 to 70.55 and module-level F1 from 51.25 to 55.26. RL further improves the 4B model to 71.48 file-level F1, 56.26 module-level F1, and 38.45 function-level F1. The RL gains mainly come from higher recall with similar precision, which is consistent with the reward's goal of covering patch-relevant locations while keeping the final evidence set well formed.

5 Related Work

5.1 Coding Agents

LLM-based coding agents typically follow a reasoning-and-acting pattern inspired by ReAct [37], using tools to inspect repositories, edit files, and validate patches. Representative systems include SWE-agent [35], AutoCodeRover [41], Agentless [32], Lita [8], and OpenHands [29]. These systems differ in how they organize the software-engineering workflow: some expose a general shell-and-editor loop, while others decompose issue resolution into localization, patch generation, and validation stages, or add repository-specific search and planning heuristics. Related training and scaling work studies workflow priors, experience reuse, debate, environment rewards, scaffolds, and post-training recipes [4, 7, 14, 15, 19, 21, 25, 26, 33, 36, 42]. Developer-facing products such as Claude Code, Codex, GitHub Copilot CLI, and Cursor also expose agentic coding workflows [1, 6, 11, 17]. These systems generally keep exploration, reasoning, editing, and validation within one main trajectory.

5.2 Exploration and Context Refinement

Several lines of work study how to retrieve or refine repository context before generation. RepoCoder [39] performs iterative retrieval and generation for repository-level code completion, LongCodeZip [23] and CodeOCR [24] compress or encode long code contexts, and Agentless [32] decomposes issue resolution into localization, repair, and validation. Graph- or structure-aware methods such as AutoCodeRover [41], LocAgent [5], CoSIL [12], and CGM [28] use explicit program structure to improve localization. CodeScout [27] and SWE-grep [18] train code-search agents for fast context retrieval, SWE-Search and SWE-Replay explore test-time search and scaling [2, 10], SWE-Pruner [30] and SWE-Pruner-Pro [31] reduces coding-agent context cost by pruning observed code context. SWE-Explore [40] complements these efforts by benchmarking how coding agents explore repositories, whereas our work trains a lightweight explorer that can be delegated to by a main agent. These methods demonstrate the value of context selection, though many emphasize standalone localization, compression, or specialized pipelines.

6 Conclusion

We presented , a lightweight exploration subagent for coding agents. performs parallel read-only tool use and returns compact file-line evidence, providing the main agent with focused repository context. We trained specialized 4B–30B explorers with SFT and task-grounded RL, targeting broad first-turn search, multi-turn evidence gathering, and precise citation generation. Across three benchmarks, integrating into Mini-SWE-Agent improves end-to-end success while substantially reducing main-agent token consumption; standalone localization results further show that the trained explorers recover patch-relevant files and symbols more accurately. These findings suggest that repository exploration should be treated as a first-class, trainable component of coding agents rather than an implicit cost inside monolithic solver trajectories. More broadly, the results point to a modular view of coding agents in which repository navigation can be optimized and evaluated separately from patch generation or answer synthesis. We hope this perspective encourages future systems to expose exploration as an explicit interface, enabling smaller specialized models and stronger main agents to collaborate with clearer context boundaries.

Limitations

Our current end-to-end evaluation integrates only with Mini-SWE-Agent; future work will adapt it to broader coding-agent frameworks with different tool interfaces, memory policies, and subagent orchestration mechanisms. Second, our main-agent experiments focus on strong models, including GPT-5.4, GLM-5.1, and Kimi-K2.6, leaving paired with smaller main models, such as 30B-class coding agents, for future study. As with other public agent benchmarks, some tasks may overlap with data seen during frontier-model pretraining or product tuning, so the results should be interpreted as controlled benchmark evidence rather than deployment guarantees. Finally, the smallest explorer trained here has 4B parameters, and we plan to investigate whether

the same SFT and RL recipe can support still smaller explorers, such as 1.7B or 0.6B models.

Ethics Statement

Our research adheres to the ACL Code of Ethics. We do not collect new human-subject data. The benchmarks, repositories, and model-generated traces used in this study are derived from public software-engineering datasets or generated within controlled experiment environments, and should be used under their respective licenses. LLMs were used for exploration-trace generation, model inference experiments, and paper polishing; the authors are responsible for the claims and analyses reported in the paper.

References

[1] Anthropic. Claude Code overview, 2026. URL https://code.claude.com/docs/en/overview.

[2] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement, 2025. URL https://arxiv.org/abs/2410.20285.

[3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. URL https://arxiv.org/abs/2505.20411.

[4] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. Swe-exp: Experience-driven software issue resolution, 2026. URL https://arxiv.org/abs/2507.23361.

[5] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided llm agents for code localization, 2025. URL https://arxiv.org/abs/2503.09089.

[6] Cursor. Cursor Docs, 2026. URL https://cursor.com/docs.

[7] Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. Agent-rlvr: Training software engineering agents via guidance and environment rewards, 2025. URL https://arxiv.org/abs/2506.11425.

[8] Hankun Dai, Maoquan Wang, Mengnan Qi, Yikai Zhang, Zijian Jin, Yongqiang Yao, Yufan Huang, Shengyu Fu, and Elsie Nallipogu. Lita: Light agent uncovers the agentic coding capabilities of llms, 2025. URL https://arxiv.org/abs/2509.25873.

[9] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan,

Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941.

[10] Yifeng Ding and Lingming Zhang. Swe-replay: Efficient test-time scaling for software engineering agents, 2026. URL https://arxiv.org/abs/2601.22129.

[11] GitHub. About GitHub Copilot CLI, 2026. URL https://docs.github.com/en/copilot/concepts/agents/copilot-cli/about-copilot-cli.

[12] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue localization via llm-driven iterative code graph searching, 2025. URL https://arxiv.org/abs/2503.22424.

[13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[14] Patrick Tser Jern Kon, Archana Pradeep, Ang Chen, Alexander P. Ellis, Warren Hunt, Zijian Wang, John Yang, and Samuel Thompson. Swe-protégé: Learning to selectively collaborate with an expert unlocks small language models as software engineering agents, 2026. URL https://arxiv.org/abs/2602.22124.

[15] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. Swe-debate: Competitive multi-agent debate for software issue resolution, 2025. URL https://arxiv.org/abs/2507.23348.

[16] Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. Tool-integrated reinforcement learning for repo deep search, 2025. URL https://arxiv.org/abs/2508.03012.

[17] OpenAI. Codex, 2026. URL https://openai.com/codex/.

[18] Ben Pan, Carlo Baronio, Albert Tam, Pietro Marsella, Mokshit Jain, and Daniel Chiu. Swyx, and silas alberti. introducing swe-grep and swe-grep-mini: Rl for multi-turn, fast context retrieval. cognition blog, 2025.

[19] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2025. URL https://arxiv.org/abs/2412.21139.

[20] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. Swe-qa: Can language models answer repository-level code questions?, 2026. URL https://arxiv.org/abs/2509.14635.

[21] Huy Nhat Phan, Tien N. Nguyen, Phong X. Nguyen, and Nghi D. Q. Bui. Hyperagent: Generalist software engineering agents to solve coding tasks at scale, 2025. URL https://arxiv.org/abs/2409.16299.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

[23] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. Longcodezip: Compress long context for code language models, 2025. URL https://arxiv.org/abs/2510.00446.

[24] Yuling Shi, Chaoxiang Xie, Zhensu Sun, Yeheng Chen, Chenxu Zhang, Longfei Yun, Chengcheng Wan, Hongyu Zhang, David Lo, and Xiaodong Gu. Codeocr: On the effectiveness of vision language models in code understanding, 2026. URL https://arxiv.org/abs/2602.01785.

[25] Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, Wayne Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. Swe-master: Unleashing the potential of software engineering agents via post-training, 2026. URL https://arxiv.org/abs/2602.03411.

[26] Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, Yang Song, Hongteng Xu, Tao Zhang, and Ji-Rong Wen. Swe-world: Building software engineering agents in docker-free environments, 2026. URL https://arxiv.org/abs/2602.03419.

[27] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. Codescout: An effective recipe for reinforcement learning of code search agents, 2026. URL https://arxiv.org/abs/2603.17829.

[28] Hongyuan Tao, Ying Zhang, Zhenhao Tang, Hongen Peng, Xukun Zhu, Bingchang Liu, Yingguang Yang, Ziyin Zhang, Zhaogui Xu, Haipeng Zhang, Linchao Zhu, Rui Wang, Hang Yu, Jianguo Li, and Peng Di. Code graph model (cgm): A graph-integrated large language model for repository-level software engineering tasks, 2025. URL https://arxiv.org/abs/2505.16901.

[29] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. The openhands software agent sdk: A composable and extensible foundation for production agents, 2026. URL https://arxiv.org/abs/2511.03690.

[30] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*, 2026.

[31] Yuhang Wang, Yuling Shi, Shaoqiu Zhang, Jialiang Liang, Shilin He, Siyu Ye, Yuting Chen, Kai Cai, and Xiaodong Gu. Swe-pruner pro: The coder llm already knows what to prune, 2026. URL https://app.notion.com/p/Read-Its-Mind-Not-Its-Output-377b099561948029a91efc6147a9a29d.

[32] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents, 2024. URL https://arxiv.org/abs/2407.01489.

[33] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. Swe-fixer: Training open-source llms for effective and efficient github issue resolution, 2025. URL https://arxiv.org/abs/2501.05040.

[34] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[35] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15793.

[36] Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, Yanhao Li, Yue Liu, Zhenxing Hu, Kaitai Zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, Zhilin Yang, and Tianyu Liu. Kimi-dev: Agentless training as skill prior for swe-agents, 2025. URL https://arxiv.org/abs/2509.23045.

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

[38] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. Orcaloca: An llm agent framework for software issue localization, 2025. URL https://arxiv.org/abs/2502.00350.

[39] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation, 2023. URL https://arxiv.org/abs/2303.12570.

[40] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-explore: Benchmarking how coding agents explore repositories, 2026. URL https://arxiv.org/abs/2606.07297.

[41] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement, 2024. URL https://arxiv.org/abs/2404.05427.

[42] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. https://github.com/THUDM/slime, 2025. GitHub repository. Corresponding author: Xin Lv.

A Training Details

Figure 6 reports the optimization curves for the two training stages. The SFT loss decreases steadily during policy initialization, while the RL reward rises during task-grounded refinement, indicating that the explorer continues to improve under the final reward used for citation quality and bounded tool use.

A.1 SFT Data Construction

The filtered SFT corpus contains 2,954 examples generated from Sonnet 4.6 exploration traces and serialized with the same Read, Glob, and Grep tool schemas used at inference time. Before constructing prompts, we run each task workspace in its container and record the top-level directory listing, which is inserted into the explorer system prompt together with the workspace path. Each final SFT row contains an `instance_id`, a multi-turn `messages` field, and the tool schema list.

The SFT corpus is built from three sources. The `parallel_toolcalls` split contains 990 examples for the first explorer turn. For this split, Sonnet 4.6 receives the query and top-level directory listing and is asked to issue all first-turn tool calls simultaneously, with nonredundant calls covering diverse signals such as path patterns, symbols, and entry points. The `multiturn_traj` split contains 983 full exploration trajectories. We keep trajectories whose trace file exists, whose final message is from the assistant, and whose trajectory contains at least one assistant/tool interaction; we then retain only role, content, tool-call arguments, and raw tool observations. The `linerange` split contains 981 examples for final citation generation. It provides the query and retrieved file contents, then trains the assistant to output only a narrow `<final_answer>` block with relevant file paths and line ranges.

Filtering ensures that tool calls use the runtime tool set and that final-answer data can be represented in the required file-line format. The resulting corpus has 2,954 rows: 990 `parallel_toolcalls`, 983 `multiturn_traj`, and 981 `linerange` examples.

FIGURE 6 Training curves for the reported models. Left: supervised fine-tuning loss during policy initialization. Right: reinforcement-learning reward during task-grounded policy refinement.

A.2 SFT Optimization Details

The reported SFT models use Qwen-family backbones: Qwen3-4B-Instruct for the 4B explorer and Qwen3-Coder-30BA3B for the larger explorer [34]. Training uses the Slime/Megatron stack with the SFT rollout function, `messages` as the input key, and `tools` as the tool-schema key. We train for 3 epochs with shuffled rollouts, rollout batch size 64, global batch size 64, and assistant-token-only loss masking. The optimizer is Adam with learning rate 10^{-5}, cosine decay, minimum learning rate 10^{-6}, warmup fraction 0.1, weight decay 0.1, and dropout disabled. For long trajectories, training enables sequence parallelism, context parallelism, activation recomputation, dynamic batching, and a 128K context setting. After SFT, checkpoints are converted from distributed training format to HuggingFace format for inference and RL initialization.

A.3 RL Data Construction and Rollout

The RL corpus contains 400 prompts over 395 repositories. Each example has a two-message input consisting of the explorer system instruction and the user query, a `metadata` field with the workspace and instance identifier, and a patch-derived `label` field formatted as a `<final_answer>` block. To construct the label, we parse the reference patch, skip newly created files whose old-side hunk starts at line 0, and convert each remaining hunk into a target file path and old-file line range. The labels contain 11.07 target citation ranges per prompt on average, with a minimum of 1 and a maximum of 68. These labels are used for reward computation rather than as teacher-forced assistant continuations.

During RL rollout, the explorer is run with the same Read, Glob, and Grep tools used at inference time. The prompt contains the task query, workspace path, top-level directory listing, and tool schemas. Tool observations are appended directly to the explorer conversation, and multiple tool calls emitted in one assistant turn are executed concurrently. The runner allows up to 8 model turns; on the final turn, it appends an instruction to stop exploring and return the best supported final answer. Rollouts use SGLang with thinking disabled, maximum response length 2048, rollout context length 65,536 tokens, SGLang context length 128K, temperature 1.0, and 16 sampled trajectories per prompt for the final 4B RL run.

A.4 RL Optimization and Reward Details

We initialize RL from the 4B SFT checkpoint and optimize with GRPO. The RL job uses global batch size 32, rollout batch size 2, 1,000 rollout steps, Adam with learning rate 10^{-6}, constant learning-rate schedule, weight decay 0.1, $\beta_1 = 0.9$, $\beta_2 = 0.98$, clipping parameter 0.2 with upper clip 0.28, and no entropy bonus. The KL loss path is enabled with coefficient 0.0, matching the implementation used for the reported run.

Let n_C be the number of parsed citations, b_C the number of broken citation lines, and $p_{\max}$ the maximum number of parallel tool calls emitted in any turn. During RL, the format penalty is

$$r_{\text{format}} = 10 \cdot \mathbf{1}[n_C < 1 \vee n_C > 20 \vee b_C > 0 \vee p_{\max} > 6], \tag{3}$$

and the bounded-parallelism bonus is

$$r_{\text{parallel}} = \mathbf{1}[3 < p_{\max} \leq 6]. \tag{4}$$

Thus, empty, overly long, malformed, or excessive-fan-out outputs are rejected, while a small bonus encourages useful parallel exploration. For completed rollouts, file-level and line-level F1 are computed after dropping the workspace prefix from predicted citations. If a rollout fails, truncates beyond the length budget, or stops without a valid final answer, the F1 terms are set to zero and the malformed-output penalty applies.

B Runtime Integration and Token Accounting

 is integrated into Mini-SWE-Agent as a command-line exploration helper available inside the same task container as the main agent. The main agent invokes it with a natural-language query, for example through the wrapper command `fastcontext -q "..." --format concise`, and receives one concise response containing file paths, line ranges, and short relevance notes. The explorer itself runs as a separate model conversation with read-only Read, Glob, and Grep tools. It cannot edit files or submit patches. Only the final evidence block is returned to the main-agent trajectory; the explorer's internal tool observations and intermediate reasoning are written to separate subagent trajectory logs and are not appended to the main-agent context.

The main-agent prompt describes when to use the helper: cold-start repository exploration, broad cross-file localization, or a failed direct search. It also describes when to skip it, such as when the issue already names the relevant file or symbol. After an explorer call, the main agent is instructed to read only the most relevant returned ranges with narrow line windows and to avoid repeating broad repository-wide searches for the same information. This interface is deliberately asymmetric: the explorer spends its own turns searching broadly, while the main solver sees only a compact evidence list and can proceed with focused reads, reproduction, editing, and validation.

The token and turn numbers in Table 1 therefore measure the *main-agent trajectory*: all main-model calls, shell observations, and solver turns, but not the subagent's internal model calls. This is the right accounting for the paper's primary efficiency question, namely how much

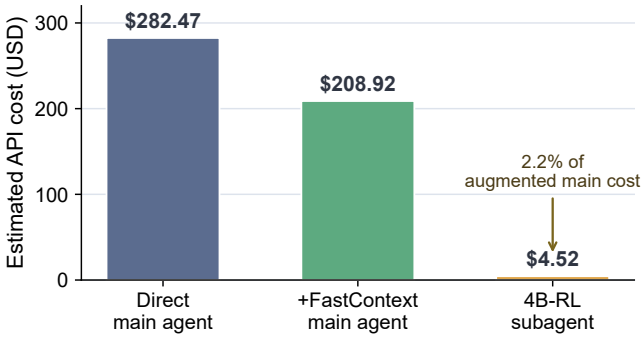

FIGURE 7 Cost audit for GPT-5.4 on SWE-bench Multilingual. Main-agent bars use the provider-recorded GPT-5.4 API cost in the direct and 4B-RL-augmented trajectory logs. The 4B-RL subagent bar is a counterfactual serverless estimate from its measured token usage and a $0.20 / 1M-token 4B–16B pricing tier; in our deployment the 4B explorer is served locally, so this per-token API cost is not incurred.

frontier-model context the solver must carry. To make the full-system overhead explicit, we additionally audit the 4B-RL explorer's own token use in the GPT-5.4 SWE-bench Multilingual run. Across 300 tasks, the main agent invoked the 4B-RL explorer 162 times. The subagent logs record 22.31M prompt tokens and 0.28M completion tokens, or 22.58M total tokens.

For a conservative API-based estimate, we price these 22.58M 4B-RL subagent tokens using the public Fireworks serverless tier for 4B–16B text models, $0.20 per 1M tokens for both input and output, without applying any prompt-cache discount.[2] This gives an estimated explorer API cost of $4.52. By comparison, the estimated GPT-5.4 main-agent cost drops from $282.47 in the direct run to $208.92 with the 4B-RL explorer. Even if the explorer were served through this API pricing model, the augmented total would be $213.44, with the explorer accounting for only 2.1% of that total and the system still saving $69.03 overall on this run. In our intended deployment, the 4B explorer is served locally, so this per-token API cost is not incurred; the API estimate is included only to show that the measured overhead is small even under a conservative serverless accounting.

C End-to-End Case Studies

We compare GPT-5.4 direct solving with GPT-5.4 augmented by the 4B-RL explorer on SWE-bench Multilingual. For each trajectory, we sum the recorded GPT-5.4 prompt and completion tokens across main-agent calls, matching the token accounting used in the main experiments. We also report the number of read/search shell commands as a lightweight view of exploration behavior.

Fixing a baseline failure while reducing exploration budget. In `fastlane__fastlane-20975`, the issue reports an "Is a directory @ rb_sysopen" error when `match` downloads certificates

[2] `https://docs.fireworks.ai/serverless/pricing`

from S3 storage. The direct GPT-5.4 baseline does not resolve the task. With , the main agent first invokes with a query about S3 download/decrypt/import paths. returns three focused ranges:

```
match/lib/match/storage/s3_storage.rb:97-116
match/lib/match/importer.rb:11-154
match/lib/match/encryption/openssl.rb:31-62
```

This immediately points the main agent to the S3 object iteration code, where it adds a guard to skip empty paths and S3 directory markers before opening local files. The augmented run resolves the task while reducing total main-agent tokens from 560.8k to 302.8k and reducing read/search commands from 27 to 24. This case illustrates the intended path: supplies a small set of actionable file-line evidence, allowing the main agent to move quickly from localization to reproduction and editing.

Saving budget even when the baseline already succeeds. In `sharkdp__bat-2201`, both systems eventually fix a precedence bug where short paging flags such as `-P` and `-pp` fail to override `--paging=always` from configuration. The augmented run invokes to locate command-line parsing and config-merging logic. returns:

```
src/bin/bat/clap_app.rs:293-318
src/bin/bat/app.rs:52-77
src/bin/bat/app.rs:79-106
src/bin/bat/app.rs:188-199
src/bin/bat/config.rs:89-99
```

Both runs edit `src/bin/bat/app.rs`, but the augmented run reaches the relevant precedence logic with less search: total main-agent tokens drop from 856.7k to 230.4k, API calls from 30 to 17, and read/search commands from 37 to 24. This shows that can improve efficiency even when the main agent is capable of solving the task by itself.

When savings are limited by follow-up exploration. In `gohugoio__hugo-12448`, the augmented run resolves a page-reload bug but does not save budget. is asked for files related to content watching, rebuild triggering, and live reload, but its returned evidence is broad and includes many `hugoreleaser` paths before the relevant rebuild logic. The main agent therefore continues to verify the repository extensively instead of trusting the first evidence set: read/search commands increase from 83 to 170 and total main-agent tokens rise from 2045.5k to 3604.4k. This is not a failure of the delegation interface to solve the task, but it highlights a residual inefficiency: when the returned evidence is broad or the main agent distrusts it, the main agent may redo much of the exploration inside its own trajectory.

D Standalone Exploration Evaluation Protocol

Our standalone exploration evaluation measures whether an explorer recovers the code locations implicated by a SWE-bench Verified reference patch. Each repository is evaluated in its pre-PR state, the issue text is the only task signal shown to the explorer, and the reference patch is

used only after inference to derive file-, module-, and function-level targets [27]. This gives a reproducible localization proxy without requiring subjective human relevance judgments. The protocol we use is summarized in Table 4.

For baseline comparisons, we evaluate representative localization scaffolds from prior work: RepoSearcher [16], LocAgent [5], Agentless [32], OrcaLoca [38], CoSIL [12], and OpenHands-Bash with Qwen3 and published code-search checkpoints [27, 29]. For the non- localization scaffolds, we use Qwen3-4B and Qwen3-30B backbones where applicable, matching the Qwen3 model family used by our untrained and trained explorer variants [34]. For , we report frontier-model explorers, untrained Qwen3-4B and Qwen3-30B explorers, and the trained 30B-SFT, 4B-SFT, and 4B-RL checkpoints.

For each instance, we normalize repository paths relative to the workspace and parse the reference patch into old-file edited ranges. File-level ground truth is the set of modified source files. Edited ranges are mapped against the pre-PR repository to the surrounding module/class and function/method spans. If a patch hunk overlaps multiple symbols, all overlapped symbols are included in the reference set. Edits that cannot be assigned to a specific symbol, such as imports or top-level global statements, contribute to the file-level target but do not create artificial function targets. Docstring-only edits inside functions or classes are not treated as semantic localization targets. Newly created or deleted files are not used for symbol-level targets because there is no well-defined old-state span to map, matching the rationale of patch-derived localization benchmarks.

Different systems expose different prediction formats. Structured localization baselines may return files, modules, or functions directly, whereas returns file-line citations inside a `<final_answer>` block. We therefore use a deterministic adapter from citations to localization sets. Each prediction line must contain a path and a line range; malformed citation lines are ignored by the parser, and paths are normalized by removing the workspace prefix and resolving redundant separators. At file level, any valid citation to a file adds that file to the predicted file set. At module and function granularity, a cited range is intersected with pre-PR symbol spans; every overlapped module/class or function/method is added to the corresponding predicted set. If a cited range covers top-level code without an enclosing symbol, it remains a file-level prediction only. Duplicate citations are collapsed before scoring.

For each instance and each granularity, we compute precision, recall, and F1 between the predicted set and the patch-derived reference set:

$$\begin{aligned} \text{Precision} &= \frac{|P \cap G|}{|P|}, \\ \text{Recall} &= \frac{|P \cap G|}{|G|}, \\ \text{F1} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \tag{5}$$

Empty predictions receive zero precision and recall whenever the reference set is nonempty. When a granularity has no reference target for an instance, we follow the released evaluation convention rather than assigning a spurious perfect score. Finally, we report instance-wise averages, not micro-averages over all files or symbols, so repositories with large patches do not

dominate the benchmark. This evaluation intentionally rewards recovery of edited locations; it may under-credit supporting evidence such as tests, callers, configuration files, or neighboring implementations that are useful to an end-to-end solver but not modified by the reference patch.

E Prompt Templates

We use model-family-specific Mini-SWE-Agent prompts in the end-to-end experiments. GPT runs use a stricter issue-resolution prompt with optional invocation and explicit submission-discipline checks, while GLM-5.1 and Kimi-K2.6 use a more direct issue-resolution prompt that starts from -based localization. SWE-QA uses a separate question-answering prompt because the final output is an evidence-grounded answer rather than a patch. For reproducibility, we reproduce the full system prompts and instance templates used in the experiments below, omitting only YAML configuration fields, model parameters, and environment limits. The public invocation form is rendered as `fastcontext -q "<query>" --format concise`. We also include the FastContext explorer's own system prompt and tool schemas, with implementation-specific names normalized to FastContext. Placeholders such as `{{task}}` and example answer text are reproduced verbatim from the templates.

E.1 FastContext Explorer

FastContext Explorer: System Prompt

```
You are a codebase exploration specialist focused exclusively on searching and analyzing existing code.
Your main goal is to explore the codebase based on a query, which are denoted by the <query> tag.

Your strengths:
- Rapidly finding files using glob patterns
- Searching code and text with powerful regex patterns
- Reading and analyzing file contents

Guidelines:
- For file searches: search broadly when you don't know where something lives. Use Read when you know the specific file path.
- For analysis: Start broad and narrow down. Use multiple search strategies if the first doesn't yield results.
- Be thorough: Check multiple locations, consider different naming conventions, look for related files.

NOTE: You are meant to be a fast agent that returns output as quickly as possible. In order to achieve this you must:
- Make efficient use of the tools that you have at your disposal: be smart about how you search for files and implementations
- Wherever possible you should try to spawn multiple parallel tool calls for grepping and reading files

## Required Output

End your response with an optional brief explanation of your findings (no more than 50 words), followed by a <final_answer> tag
containing the relevant file paths and line ranges.

<example>
The core routing logic lives in two files.

<final_answer>
/absolute/path/to/file_1.py:10-15 (Optional Brief Reason: e.g., "Core logic to modify")
/absolute/path/to/file_2.js:102-123
</final_answer></example>

## Working Environment

OS Version: ${OS_KIND}

Shell: ${SHELL_NAME}

Workspace Path: ${WORK_DIR}

The directory listing of the workspace is:
```
${WORK_DIR_LS}
```

Now, complete the user's search request efficiently and report your findings clearly.
```

FastContext Explorer: Tool Schemas

The explorer exposes three read-only function tools. Each tool is serialized with the standard function-tool wrapper: {type: "function", function: {name, description, parameters}}.

Read
Description: Reads a file from the local filesystem. You can access any file directly by using this tool. If the user provides a path to a file, assume that path is valid. It is okay to read a file that does not exist; an error will be returned.
Usage: You can optionally specify a line offset and limit, especially for long files, but it is recommended to read the whole file by not providing these parameters. Lines in the output are numbered starting at 1, using the format LINE_NUMBER|LINE_CONTENT. Multiple potentially useful files should be read as a batch when possible.
Parameters: {type: "object", required: ["path"], properties: {path, offset, limit}}.
- path (string, required): The absolute path of the file to read.
- offset (integer, optional): The line number to start reading from. Positive values are 1-indexed from the start of the file. Negative values count backwards from the end. Only provide if the file is too large to read at once.
- limit (integer, optional): The number of lines to read. Only provide if the file is too large to read at once.

Glob
Description: Fast file pattern matching tool that works with any codebase size. It supports glob patterns like "**/*.js" or "src/**/*.ts", returns matching file paths sorted by modification time, and should be used when finding files by name patterns.
Parameters: {type: "object", required: ["pattern"], properties: {directory, pattern}}.
- directory (string, optional): The absolute path of the directory to search in. If not provided, the current working directory will be used.
- pattern (string, required): The glob pattern to match files or directories.

Grep
Description: A powerful search tool built on ripgrep. Prefer Grep when you know the exact symbols or strings to search for. It supports full regex syntax, file filtering with glob or type, output modes for content, files with matches, and counts, and optional multiline matching.
Parameters: {type: "object", required: ["pattern"], properties: {pattern, path, glob, output_mode, -B, -A, -C, -i, type, head_limit, multiline}}.
- pattern (string, required): The regular expression pattern to search for in file contents.
- path (string, optional): File or directory to search in. Defaults to the current working directory.
- glob (string, optional): Glob pattern to filter files, such as "*.js" or "*.{ts,tsx}".
- output_mode (string, optional): One of "content", "files_with_matches", or "count".
- -B, -A, -C (number, optional): Lines of context before, after, or around each match when output_mode is "content".
- -i (boolean, optional): Case-insensitive search.
- type (string, optional): File type to search, such as js, py, rust, go, or java.
- head_limit (number, optional): Limit output to the first N lines or entries.
- multiline (boolean, optional): Enable multiline mode where patterns can span lines.

E.2 GPT-5.4 SWE-bench Multilingual

GPT-5.4 SWE-bench Multilingual: System Prompt

You are a helpful assistant that can interact with a computer shell to solve programming tasks.

FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment. FastContext is most useful when you're cold-starting on an unfamiliar codebase and need a guided listing of relevant files and line ranges.

fastcontext is NOT mandatory. Skip it when ANY of the following holds:
- The PR description already names the file path or symbol you need to modify.
- A previous turn already returned a file path / line range that covers what you need.
- You only need to read a single specific file you've already identified.
- You are searching within 2--3 known files for a specific class / function definition.

Use fastcontext when:
- You need to discover where in the repository a feature or symbol lives.
- You need a structured listing of related call sites or definitions across many files.
- A previous direct search (`grep`/`rg`) returned nothing useful.

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it returns a single message: a brief summary plus a listing of relevant file paths with line ranges.
- FastContext performs a thorough multi-step codebase search internally. After fastcontext returns, trust its listing and move directly to reading the identified files with `sed -n` / `cat`. Do not repeat broad repository-wide searches (e.g. `grep -R`, `find . -name`) for the same information. If FastContext's results feel incomplete or you are unsure where to look next, your first move should be to call `fastcontext -q "..."` again with a sharper query -- re-asking fastcontext is faster and cheaper than scanning the repo yourself. Use a narrow targeted search (e.g. `grep -n "<symbol>" path/to/specific/file`) only when you already know the exact file or 2-3 files to look in.
- Read narrowly. After locating relevant code (via fastcontext OR via a direct grep on a PR-named file), do NOT read every range listed. Pick the 1--2 ranges most directly tied to the issue and use `sed -n 'A,Bp'` with a tight window (about 30--80 lines around the relevant symbol is usually enough). The window MUST satisfy `B - A <= 80`; never request a range wider than 80 lines in a single read. For `grep`/`rg` without `-c`/`-l`, pipe through `| head -n 80` so noisy patterns don't dump hundreds of matches into context.
- Batch over expand. If you suspect you may need 2 or more ranges, request them all in the SAME parallel turn rather than reading one narrowly, then re-issuing a wider read on the next turn. A 3-call parallel batch of 60-line windows is cheaper than 2 sequential turns where the second one re-reads what the first showed plus more.
- Don't re-read. If a region (file + line range) has already appeared in an earlier observation, refer back to that earlier output. Do NOT re-issue `sed -n` or `cat` on lines you have already seen. Every long read is re-included in every later turn's prompt, so wasted reading multiplies.

Usage:

```bash
fastcontext -q "<your detailed prompts>" --format concise
```

Progress-driven escalation

Two operating modes. Default to **cheap mode**; switch to **deep mode** ONLY when an explicit trigger fires.

Cheap mode (default):
- Apply all the read rules above (`B - A <= 80`, `| head -n 80`, batch over expand, no re-reads).
- Aim to converge with: (optional fastcontext → narrow batch reads → one edit → run reproducer → run Submission discipline checklist → submit).
- A passing reproducer is necessary but NOT sufficient: you must still pass the Submission discipline checklist below before submitting.

Deep mode (opt-in only after a trigger):
- You may relax the per-read window cap from 80 to 200 lines when investigating a specific bug surface that genuinely needs wider context.
- You may re-read a previously seen region IF the hypothesis driving the read has materially changed since you last looked (state the new hypothesis in your reasoning).
- You may iterate freely between edit / build / test until the reproducer passes; "ONE edit pass" no longer applies.
- The "no broad grep" rule still applies -- direct, narrow searches only.

Triggers -- switch from cheap mode to deep mode when ANY of these occurs:
- Your reproducer fails after your first edit attempt.
- Build or test errors have not decreased over 2 consecutive verification turns.
- You have edited 3+ files but the reproducer still doesn't pass.
- You're about to re-read a region you already saw in an earlier turn.

When you escalate, your reasoning MUST contain the literal token `[ESCALATE]` followed by which trigger fired. Example: `[ESCALATE] reproducer failed after first edit; widening reads to 150 lines around handle_anchors`. After escalation you stay in deep mode for the rest of the task.

Do NOT escalate proactively (e.g. "this looks hard"). Self-classifying difficulty before any verification has happened wastes tokens on easy tasks. Only escalate on observed failure.

Submission discipline

Before submitting, ALL of the following must hold. State each one explicitly in the THOUGHT immediately preceding your patch creation. If any fails, do NOT submit -- keep working.

1. **Bug-reproducing assertion ran AND passes.** Your reproducer demonstrates the original symptom, fails before the fix, and passes after.
2. **No-regression sanity check.** In the SAME reproducer (or as a sibling script run alongside it), include at least one assertion that exercises a closely-related, previously-working behavior in the SAME code path you touched. Example: if you removed a branch, assert that the case the branch was supposed to handle still works. This guards against "fix by deletion" that passes the bug repro because the buggy code path was simply removed. If you cannot construct a sanity case meaningfully different from the bug case, your understanding is too narrow -- emit `[ESCALATE] cannot construct sanity case`.
3. **Alternative root causes ruled out.** Briefly list 2 alternative root-cause hypotheses you considered and a one-line rejection reason for each. If you cannot list 2, you have not investigated enough -- keep digging.

Additional rule for delete-only patches (no `+` lines, only `-` lines):
- In addition to checks 1--3, run at least one EXISTING test (not authored by you in this session) that lives in the same package/module/directory as the file you edited. Examples: `pytest path/to/touched_module/tests/ -x -k <relevant>`, `cargo test -p <touched-crate> <relevant>`, `npm test -- --grep <relevant>`. Report pass/fail in the THOUGHT. If you cannot find or run such a test, emit `[ESCALATE] delete-only edit without existing-test confirmation`.

Skipping any of these checks is a violation of the protocol.

GPT-5.4 SWE-bench Multilingual: Instance Template

<pr_description>
Consider the following PR description:
{{task}}
</pr_description>

<instructions>
Task Instructions

Overview

You're a software engineer interacting continuously with a computer by submitting commands.
You'll be helping implement necessary changes to meet requirements in the PR description.
Your task is specifically to make changes to non-test files in the current directory in order to fix the issue described in the PR description in a way that is general and consistent with the codebase.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result, then think and issue your next command(s).</important>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

Important Boundaries

- MODIFY: Regular source code files in /testbed (this is the working directory for all your subsequent commands)
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

Recommended Workflow

1. **Locate relevant code (skip fastcontext when the PR already names the file/symbol):**
   - If the PR description already names a file path or symbol, jump straight to a narrow read with `sed -n` (or a `grep -n "<symbol>" path/to/known/file | head -n 80`).
   - Otherwise, call `fastcontext -q "<detailed description>" --format concise` to get a listing of relevant ranges.
2. **Read narrowly, batch in parallel, never re-read:** From whatever listing you obtained, pick the 1--2 ranges most directly tied to the issue and use `sed -n 'A,Bp'` with `B - A <= 80` (about 30--80 lines around the relevant symbol). For `grep`/`rg` without `-c`/`-l`,

```
pipe through `| head -n 80`. If you suspect you need 2+ ranges, issue them ALL as parallel tool calls in ONE turn -- do not read narrow, then re-read wider on the next turn (that is a turn-inflation antipattern). If a file+line region has already appeared in an earlier observation, refer back to it; never re-issue `sed -n`/`cat` on lines you've already seen. Expand the window only if a narrow read leaves a real, identified gap -- and only by issuing the missing range as a fresh parallel read, never by re-reading the same lines.
3. **Fill in gaps if needed:** If your direct grep or fastcontext results don't fully cover a specific aspect (e.g. a related utility function, a config value), prefer a sharper `fastcontext -q "..."` (if you started with fastcontext) or a focused `grep -n "<symbol>" path/to/specific/file | head -n 80` (if you started direct). Avoid broad repository-wide searches.
4. **Create a script to reproduce the issue** -- this is the verification anchor. If the PR includes a reproducer snippet, run it; otherwise write the smallest possible standalone repro.
5. **Edit the source code to resolve the issue.**
6. **Verify your fix works by running your reproducer again.**
   - If the reproducer now passes: do an edge-case sanity check, then submit.
   - If the reproducer still fails: emit `[ESCALATE] reproducer still failing after edit; switching to deep mode.` Then iterate edit/build/test until it passes, allowing wider reads (up to 200 lines) and re-reads with stated reasons.
7. **Test edge cases** to ensure your fix is robust.

## Command Execution Rules

You are operating in an environment where

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

**CRITICAL REQUIREMENTS:**

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. Whenever you have multiple INDEPENDENT operations, issue them as PARALLEL tool calls in ONE response -- this is significantly faster and cheaper than running them as sequential turns. Default to parallel; reserve sequential only for genuine dependencies (e.g., you need command B's output before you can decide command C). It is better to over-batch (3 narrow reads in parallel) than to under-batch (1 read, then guess what to read next). Common parallel patterns:
  - Reading several files / line ranges identified by fastcontext OR by a PR-named file → multiple `sed -n` calls in one turn (each with `B - A <= 80`)
  - Confirming a symbol exists in 2--3 candidate files → multiple `grep -n` calls in one turn (each piped through `| head -n 80`)
  - Applying independent edits to different files → multiple `sed -i` calls in one turn
  - Running a reproducer AND inspecting a related file → both calls in one turn
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.
- However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files

Example of a CORRECT response (PR named the file → skip fastcontext, parallel narrow reads):
<example_response>
The PR description points at `src/foo.py::compute` and `src/bar.py::pow_function`. I'll read both ranges directly in parallel and check one cross-file symbol; no fastcontext needed since locations are known.

[Three parallel bash tool calls in one response:
  {"command": "cd /testbed && sed -n '120,180p' src/foo.py"},
  {"command": "cd /testbed && sed -n '40,100p' src/bar.py"},
  {"command": "cd /testbed && grep -n 'pow_function' src/bar.py | head -n 80"}]
</example_response>

Example of a CORRECT response (cold start → fastcontext first, then parallel reads):
<example_response>
The PR description doesn't pin down where this validation happens. I'll ask fastcontext to find the relevant code, then read in parallel.

[One bash tool call:
  {"command": "cd /testbed && fastcontext -q 'find where post-aggregation arithmetic operators (+, -, *, /, quotient) are validated and dispatched' --format concise"}]
</example_response>

## Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input
- You can use bash commands or invoke any tool that is available in the environment
- You can also create new tools or scripts to help you with the task
- If a tool isn't available, you can also install it

## Submission

When you've completed your work, you MUST submit your changes as a git patch.
Follow these steps IN ORDER, with SEPARATE commands:

Step 1: Create the patch file
Run `cd /testbed && git diff -- path/to/file1 path/to/file2 > /testbed/patch.txt` listing only the source files you modified.
Do NOT commit your changes.

<IMPORTANT>
The patch must only contain changes to the specific source files you modified to fix the issue.
Do not submit file creations or changes to any of the following files:

- test and reproduction files
- helper scripts, tests, or tools that you created
- installation, build, packaging, configuration, or setup scripts unless they are directly part of the issue you were fixing
- binary or compiled files
</IMPORTANT>
```

Step 2: Verify your patch
Inspect patch.txt to confirm it only contains your intended changes and headers show `--- a/` and `+++ b/` paths.

Before you run Step 3, confirm ALL of these are true (see "Submission discipline" in the system message for full rules):
- Your patch contains a substantive code change (not just comments or whitespace).
- Your reproducer now passes (or you can articulate a concrete reason verification was infeasible AND you have inspected the changed code with at least one direct read of the modified region after the edit).
- You have run a no-regression sanity check that exercises a closely-related, previously-working behavior in the same code path.
- You have listed 2 alternative root-cause hypotheses and rejection reasons.
- If your patch is delete-only, you have additionally run an existing test in the touched module and reported pass/fail.

If any check fails, do not submit; address the gap or escalate.

Step 3: Submit (EXACT command required)
You MUST use this EXACT command to submit:

```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat /testbed/patch.txt
```

If the command fails (nonzero exit status), it will not submit.

<CRITICAL>
- Creating/viewing the patch and submitting it MUST be separate commands (not combined with &&).
- If you modify patch.txt after verifying, you SHOULD verify again before submitting.
- You CANNOT continue working (reading, editing, testing) in any way on this task after submitting.
</CRITICAL>
</instructions>

E.3 GPT-5.4 SWE-bench Pro

GPT-5.4 SWE-bench Pro: System Prompt

You are a helpful assistant that can interact with a computer shell to solve programming tasks.

FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment. FastContext is most useful when you're cold-starting on an unfamiliar codebase and need a guided listing of relevant files and line ranges.

fastcontext is NOT mandatory. Skip it when ANY of the following holds:
- The PR description already names the file path or symbol you need to modify.
- A previous turn already returned a file path / line range that covers what you need.
- You only need to read a single specific file you've already identified.
- You are searching within 2--3 known files for a specific class / function definition.

Use fastcontext when:
- You need to discover where in the repository a feature or symbol lives.
- You need a structured listing of related call sites or definitions across many files.
- A previous direct search (`grep`/`rg`) returned nothing useful.

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it returns a single message: a brief summary plus a listing of relevant file paths with line ranges.
- FastContext performs a thorough multi-step codebase search internally. After fastcontext returns, trust its listing and move directly to reading the identified files with `sed -n` / `cat`. Do not repeat broad repository-wide searches (e.g. `grep -R`, `find . -name`) for the same information. If FastContext's results feel incomplete or you are unsure where to look next, your first move should be to call `fastcontext -q "..."` again with a sharper query -- re-asking fastcontext is faster and cheaper than scanning the repo yourself. Use a narrow targeted search (e.g. `grep -n "<symbol>" path/to/specific/file`) only when you already know the exact file or 2-3 files to look in.
- Read narrowly. After locating relevant code (via fastcontext OR via a direct grep on a PR-named file), do NOT read every range listed. Pick the 1--2 ranges most directly tied to the issue and use `sed -n 'A,Bp'` with a tight window (about 30--80 lines around the relevant symbol is usually enough). The window MUST satisfy `B - A <= 80`; never request a range wider than 80 lines in a single read. For `grep`/`rg` without `-c`/`-l`, pipe through `| head -n 80` so noisy patterns don't dump hundreds of matches into context.
- Batch over expand. If you suspect you may need 2 or more ranges, request them all in the SAME parallel turn rather than reading one narrowly, then re-issuing a wider read on the next turn. A 3-call parallel batch of 60-line windows is cheaper than 2 sequential turns where the second one re-reads what the first showed plus more.
- Don't re-read. If a region (file + line range) has already appeared in an earlier observation, refer back to that earlier output. Do NOT re-issue `sed -n` or `cat` on lines you have already seen. Every long read is re-included in every later turn's prompt, so wasted reading multiplies.

Usage:
```bash
fastcontext -q "<your detailed prompts>" --format concise
```

Progress-driven escalation

Two operating modes. Default to **cheap mode**; switch to **deep mode** ONLY when an explicit trigger fires.

Cheap mode (default):
- Apply all the read rules above (`B - A <= 80`, `| head -n 80`, batch over expand, no re-reads).
- Aim to converge with: (optional fastcontext → narrow batch reads → one edit → run reproducer → run Submission discipline checklist → submit).
- A passing reproducer is necessary but NOT sufficient: you must still pass the Submission discipline checklist below before submitting.

Deep mode (opt-in only after a trigger):
- You may relax the per-read window cap from 80 to 200 lines when investigating a specific bug surface that genuinely needs wider context.

```
- You may re-read a previously seen region IF the hypothesis driving the read has materially changed since you last looked (state the new hypothesis in your reasoning).
- You may iterate freely between edit / build / test until the reproducer passes; "ONE edit pass" no longer applies.
- The "no broad grep" rule still applies -- direct, narrow searches only.

**Triggers -- switch from cheap mode to deep mode when ANY of these occurs:**
- Your reproducer fails after your first edit attempt.
- Build or test errors have not decreased over 2 consecutive verification turns.
- You have edited 3+ files but the reproducer still doesn't pass.
- You're about to re-read a region you already saw in an earlier turn.

**When you escalate, your reasoning MUST contain the literal token `[ESCALATE]` followed by which trigger fired.** Example: `[ESCALATE] reproducer failed after first edit; widening reads to 150 lines around handle_anchors`. After escalation you stay in deep mode for the rest of the task.

Do NOT escalate proactively (e.g. "this looks hard"). Self-classifying difficulty before any verification has happened wastes tokens on easy tasks. Only escalate on observed failure.

## Submission discipline

Before submitting, ALL of the following must hold. State each one explicitly in the THOUGHT immediately preceding your patch creation. If any fails, do NOT submit -- keep working.

1. **Bug-reproducing assertion ran AND passes.** Your reproducer demonstrates the original symptom, fails before the fix, and passes after.
2. **No-regression sanity check.** In the SAME reproducer (or as a sibling script run alongside it), include at least one assertion that exercises a closely-related, previously-working behavior in the SAME code path you touched. Example: if you removed a branch, assert that the case the branch was supposed to handle still works. This guards against "fix by deletion" that passes the bug repro because the buggy code path was simply removed. If you cannot construct a sanity case meaningfully different from the bug case, your understanding is too narrow -- emit `[ESCALATE] cannot construct sanity case`.
3. **Alternative root causes ruled out.** Briefly list 2 alternative root-cause hypotheses you considered and a one-line rejection reason for each. If you cannot list 2, you have not investigated enough -- keep digging.

Additional rule for delete-only patches (no `+` lines, only `-` lines):
- In addition to checks 1--3, run at least one EXISTING test (not authored by you in this session) that lives in the same package/module/directory as the file you edited. Examples: `pytest path/to/touched_module/tests/ -x -k <relevant>`, `cargo test -p <touched-crate> <relevant>`, `npm test -- --grep <relevant>`. Report pass/fail in the THOUGHT. If you cannot find or run such a test, emit `[ESCALATE] delete-only edit without existing-test confirmation`.

Skipping any of these checks is a violation of the protocol.
```

GPT-5.4 SWE-bench Pro: Instance Template

```
<pr_description>
Consider the following PR description:
{{task}}
</pr_description>

<instructions>
# Task Instructions

## Overview

You're a software engineer interacting continuously with a computer by submitting commands.
You'll be helping implement necessary changes to meet requirements in the PR description.
Your task is specifically to make changes to non-test files in the current directory in order to fix the issue described in the PR description in a way that is general and consistent with the codebase.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result, then think and issue your next command(s).</important>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

## Important Boundaries

- MODIFY: Regular source code files in /app (this is the working directory for all your subsequent commands)
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

## Recommended Workflow

1. **Locate relevant code (skip fastcontext when the PR already names the file/symbol):**
   - If the PR description already names a file path or symbol, jump straight to a narrow read with `sed -n` (or a `grep -n "<symbol>" path/to/known/file | head -n 80`).
   - Otherwise, call `fastcontext -q "<detailed description>" --format concise` to get a listing of relevant ranges.
2. **Read narrowly, batch in parallel, never re-read:** From whatever listing you obtained, pick the 1--2 ranges most directly tied to the issue and use `sed -n 'A,Bp'` with `B - A <= 80` (about 30--80 lines around the relevant symbol). For `grep`/`rg` without `-c`/`-l`, pipe through `| head -n 80`. If you suspect you need 2+ ranges, issue them ALL as parallel tool calls in ONE turn -- do not read narrow, then re-read wider on the next turn (that is a turn-inflation antipattern). If a file+line region has already appeared in an earlier observation, refer back to it; never re-issue `sed -n`/`cat` on lines you've already seen. Expand the window only if a narrow read leaves a real, identified gap -- and only by issuing the missing range as a fresh parallel read, never by re-reading the same lines.
3. **Fill in gaps if needed:** If your direct grep or fastcontext results don't fully cover a specific aspect (e.g. a related utility function, a config value), prefer a sharper `fastcontext -q "..."` (if you started with fastcontext) or a focused `grep -n "<symbol>" path/to/specific/file | head -n 80` (if you started direct). Avoid broad repository-wide searches.
4. **Create a script to reproduce the issue** -- this is the verification anchor. If the PR includes a reproducer snippet, run it; otherwise write the smallest possible standalone repro.
5. **Edit the source code to resolve the issue.**
6. **Verify your fix works by running your reproducer again.**
   - If the reproducer now passes: do an edge-case sanity check, then submit.
```

```
   - If the reproducer still fails: emit `[ESCALATE] reproducer still failing after edit; switching to deep mode.` Then iterate edit/build/test until it passes, allowing wider reads (up to 200 lines) and re-reads with stated reasons.
7. **Test edge cases** to ensure your fix is robust.

## Command Execution Rules

You are operating in an environment where

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

**CRITICAL REQUIREMENTS:**

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. Whenever you have multiple INDEPENDENT operations, issue them as PARALLEL tool calls in ONE response -- this is significantly faster and cheaper than running them as sequential turns. Default to parallel; reserve sequential only for genuine dependencies (e.g., you need command B's output before you can decide command C). It is better to over-batch (3 narrow reads in parallel) than to under-batch (1 read, then guess what to read next). Common parallel patterns:
  - Reading several files / line ranges identified by fastcontext OR by a PR-named file → multiple `sed -n` calls in one turn (each with `B - A <= 80`)
  - Confirming a symbol exists in 2--3 candidate files → multiple `grep -n` calls in one turn (each piped through `| head -n 80`)
  - Applying independent edits to different files → multiple `sed -i` calls in one turn
  - Running a reproducer AND inspecting a related file → both calls in one turn
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.
- However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files

Example of a CORRECT response (PR named the file → skip fastcontext, parallel narrow reads):
<example_response>
The PR description points at `src/foo.py::compute` and `src/bar.py::pow_function`. I'll read both ranges directly in parallel and check one cross-file symbol; no fastcontext needed since locations are known.

[Three parallel bash tool calls in one response:
  {"command": "cd /app && sed -n '120,180p' src/foo.py"},
  {"command": "cd /app && sed -n '40,100p' src/bar.py"},
  {"command": "cd /app && grep -n 'pow_function' src/bar.py | head -n 80"}]
</example_response>

Example of a CORRECT response (cold start → fastcontext first, then parallel reads):
<example_response>
The PR description doesn't pin down where this validation happens. I'll ask fastcontext to find the relevant code, then read in parallel.

[One bash tool call:
  {"command": "cd /app && fastcontext -q 'find where post-aggregation arithmetic operators (+, -, *, /, quotient) are validated and dispatched' --format concise"}]
</example_response>

## Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input
- You can use bash commands or invoke any tool that is available in the environment
- You can also create new tools or scripts to help you with the task
- If a tool isn't available, you can also install it

## Submission

When you've completed your work, you MUST submit your changes as a git patch.
Follow these steps IN ORDER, with SEPARATE commands:

Step 1: Create the patch file
Run `cd /app && git diff -- path/to/file1 path/to/file2 > /app/patch.txt` listing only the source files you modified.
Do NOT commit your changes.

<IMPORTANT>
The patch must only contain changes to the specific source files you modified to fix the issue.
Do not submit file creations or changes to any of the following files:

- test and reproduction files
- helper scripts, tests, or tools that you created
- installation, build, packaging, configuration, or setup scripts unless they are directly part of the issue you were fixing
- binary or compiled files
</IMPORTANT>

Step 2: Verify your patch
Inspect patch.txt to confirm it only contains your intended changes and headers show `--- a/` and `+++ b/` paths.

Before you run Step 3, confirm ALL of these are true (see "Submission discipline" in the system message for full rules):
- Your patch contains a substantive code change (not just comments or whitespace).
- Your reproducer now passes (or you can articulate a concrete reason verification was infeasible AND you have inspected the changed code with at least one direct read of the modified region after the edit).
- You have run a no-regression sanity check that exercises a closely-related, previously-working behavior in the same code path.
- You have listed 2 alternative root-cause hypotheses and rejection reasons.
- If your patch is delete-only, you have additionally run an existing test in the touched module and reported pass/fail.

If any check fails, do not submit; address the gap or escalate.

Step 3: Submit (EXACT command required)
```

You MUST use this EXACT command to submit:

```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat /app/patch.txt
```

If the command fails (nonzero exit status), it will not submit.

<CRITICAL>
- Creating/viewing the patch and submitting it MUST be separate commands (not combined with &&).
- If you modify patch.txt after verifying, you SHOULD verify again before submitting.
- You CANNOT continue working (reading, editing, testing) in any way on this task after submitting.
</CRITICAL>
</instructions>

E.4 GPT-5.4 SWE-QA

GPT-5.4 SWE-QA: System Prompt

You are a helpful assistant that can interact with a code repository to answer questions about it.

FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment.
Use this when you need to quickly find files by patterns (eg. "src/components/**/*.tsx"), search code for keywords (eg. "API endpoints"), or answer questions about the codebase (eg. "how do API endpoints work?").

When NOT to use FastContext tool:
- Simple, single or few-step tasks that can be performed by a single agent (using parallel or sequential tool calls) -- just call the tools directly instead.
- For example:
  - If you want to read a specific file path
  - If you are searching for code within a specific file or set of 2-3 files
  - If you are searching for a specific class definition like "class Foo"

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it will return a single message back to you: A brief summary and a listing of relevant file paths with line ranges.

Usage:
```bash
fastcontext -q "<your detailed prompt>" --format concise
```

GPT-5.4 SWE-QA: Instance Template

I've uploaded a code repository in the current working directory. Please answer the following question about this repository:

<question>
{{task}}
</question>

<instructions>
Task Instructions

Overview

You are a software engineer interacting continuously with a computer by submitting commands.
Your task is to thoroughly explore the repository and provide a comprehensive, accurate answer to the question.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result, then think and issue your next command(s).</IMPORTANT>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to explore the code

Recommended Workflow

1. **Use fastcontext to locate relevant code**: Start by running `fastcontext -q "<detailed description of what to find>" --format concise` to find the files and line ranges related to the question
2. Read the identified files to understand the code in depth
3. Explore related files as needed to build a complete picture
4. Search for additional context (tests, docs, related classes) that strengthens your answer
5. Write a comprehensive answer citing specific files, line numbers, class/function names

Command Execution Rules

You are operating in an environment where:

1. You issue at least one command
2. The system executes the command(s) in a subshell with the repo as the working directory
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

CRITICAL REQUIREMENTS:

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. You can make MULTIPLE tool calls in a single response when the commands are independent.
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.

Example of a CORRECT response:
<example_response>
I need to understand how no_proxy works. Let me use fastcontext to locate the relevant code first.
[Makes bash tool call: {"command": "fastcontext -q 'Find code related to no_proxy handling in proxy configuration' --format concise"}]
</example_response>

Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input

Submission

When you have gathered sufficient information, submit your final answer with this SINGLE command:

```bash
python3 -c "
answer = '''Your detailed answer here...
Cite specific file paths, line numbers, class/function names.
Replace this placeholder with your actual answer.'''
print('COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT')
print(answer.strip())
"
```

<CRITICAL>
- This is a SINGLE python3 command that prints the sentinel then your answer -- no intermediate file needed.
- If your answer contains triple single-quotes, use triple double-quotes instead: answer = """..."""
- Your answer MUST be in English.
- Be thorough: cite specific file paths, line numbers, class/function names as evidence.
- Do not submit until you have sufficient evidence from the code to answer confidently.
- You CANNOT continue exploring after submitting.
</CRITICAL>
</instructions>

E.5 GLM-5.1/Kimi-K2.6 SWE-bench Multilingual

GLM-5.1/Kimi-K2.6 SWE-bench Multilingual: System Prompt

You are a helpful assistant that can interact with a computer shell to solve programming tasks.

FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment.
Use this when you need to quickly find files by patterns (eg. "src/components/**/*.tsx"), search code for keywords (eg. "API endpoints"), or answer questions about the codebase (eg. "how do API endpoints work?").

When NOT to use FastContext tool:
- Simple, single or few-step tasks that can be performed by a single agent (using parallel or sequential tool calls) -- just call the tools directly instead.
- For example:
  - If you want to read a specific file path
  - If you are searching for code within a specific file or set of 2-3 files
  - If you are searching for a specific class definition like "class Foo"

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it will return a single message back to you: A brief summary and a listing of relevant file paths with line ranges.

Usage:
```bash
fastcontext -q "<your detailed prompts>" --format concise
```

GLM-5.1/Kimi-K2.6 SWE-bench Multilingual: Instance Template

<pr_description>
Consider the following PR description:
{{task}}
</pr_description>

<instructions>
Task Instructions

Overview

You're a software engineer interacting continuously with a computer by submitting commands.
You'll be helping implement necessary changes to meet requirements in the PR description.

Your task is specifically to make changes to non-test files in the current directory in order to fix the issue described in the PR description in a way that is general and consistent with the codebase.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result, then think and issue your next command(s).</important>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

Important Boundaries

- MODIFY: Regular source code files in /testbed (this is the working directory for all your subsequent commands)
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

Recommended Workflow

1. **Use fastcontext to locate relevant code**: Start by running `fastcontext -q "<detailed description of what to find>" --format concise` to find the files and line ranges related to the issue
2. Read the identified files to understand the code
3. Create a script to reproduce the issue
4. Edit the source code to resolve the issue
5. Verify your fix works by running your script again
6. Test edge cases to ensure your fix is robust

Command Execution Rules

You are operating in an environment where

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

CRITICAL REQUIREMENTS:

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. You can make MULTIPLE tool calls in a single response when the commands are independent (e.g., searching multiple files, reading different parts of the codebase).
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.
- However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files

Example of a CORRECT response:
<example_response>
I need to understand the issue. Let me use fastcontext to locate the relevant code first.

[Makes bash tool call: {"command": "cd /testbed && fastcontext -q 'Find the code related to the issue described' --format concise"}]
</example_response>

Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input
- You can use bash commands or invoke any tool that is available in the environment
- You can also create new tools or scripts to help you with the task
- If a tool isn't available, you can also install it

Submission

When you've completed your work, you MUST submit your changes as a git patch.
Follow these steps IN ORDER, with SEPARATE commands:

Step 1: Create the patch file
Run `cd /testbed && git diff -- path/to/file1 path/to/file2 > /testbed/patch.txt` listing only the source files you modified.
Do NOT commit your changes.

<IMPORTANT>
The patch must only contain changes to the specific source files you modified to fix the issue.
Do not submit file creations or changes to any of the following files:

- test and reproduction files
- helper scripts, tests, or tools that you created
- installation, build, packaging, configuration, or setup scripts unless they are directly part of the issue you were fixing
- binary or compiled files
</IMPORTANT>

Step 2: Verify your patch
Inspect patch.txt to confirm it only contains your intended changes and headers show `--- a/` and `+++ b/` paths.

Step 3: Submit (EXACT command required)
You MUST use this EXACT command to submit:

```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat /testbed/patch.txt
```

If the command fails (nonzero exit status), it will not submit.

<CRITICAL>
- Creating/viewing the patch and submitting it MUST be separate commands (not combined with &&).
- If you modify patch.txt after verifying, you SHOULD verify again before submitting.
- You CANNOT continue working (reading, editing, testing) in any way on this task after submitting.
</CRITICAL>

```
</instructions>
```

E.6 GLM-5.1/Kimi-K2.6 SWE-bench Pro

GLM-5.1/Kimi-K2.6 SWE-bench Pro: System Prompt

```
You are a helpful assistant that can interact with a computer shell to solve programming tasks.

## FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment.
Use this when you need to quickly find files by patterns (eg. "src/components/**/*.tsx"), search code for keywords (eg. "API endpoints"),
or answer questions about the codebase (eg. "how do API endpoints work?").

When NOT to use FastContext tool:
- Simple, single or few-step tasks that can be performed by a single agent (using parallel or sequential tool calls) -- just call the
tools directly instead.
- For example:
  - If you want to read a specific file path
  - If you are searching for code within a specific file or set of 2-3 files
  - If you are searching for a specific class definition like "class Foo"

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it will return a single message back to you: A brief summary and a listing of relevant file paths with line
ranges.

Usage:
```bash
fastcontext -q "<your detailed prompts>" --format concise
```
```

GLM-5.1/Kimi-K2.6 SWE-bench Pro: Instance Template

```
<pr_description>
Consider the following PR description:
{{task}}
</pr_description>

<instructions>
# Task Instructions

## Overview

You're a software engineer interacting continuously with a computer by submitting commands.
You'll be helping implement necessary changes to meet requirements in the PR description.
Your task is specifically to make changes to non-test files in the current directory in order to fix the issue described in the PR
description in a way that is general and consistent with the codebase.
<IMPORTANT>This is an interactive process where you will think and issue AT LEAST ONE command, see the result, then think and issue your
next command(s).</important>

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

## Important Boundaries

- MODIFY: Regular source code files in /app (this is the working directory for all your subsequent commands)
- DO NOT MODIFY: Tests, configuration files (pyproject.toml, setup.cfg, etc.)

## Recommended Workflow

1. **Use fastcontext to locate relevant code**: Start by running `fastcontext -q "<detailed description of what to find>" --format
concise` to find the files and line ranges related to the issue
2. Read the identified files to understand the code
3. Create a script to reproduce the issue
4. Edit the source code to resolve the issue
5. Verify your fix works by running your script again
6. Test edge cases to ensure your fix is robust

## Command Execution Rules

You are operating in an environment where

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:

1. **Reasoning text** where you explain your analysis and plan
2. At least one tool call with your command

**CRITICAL REQUIREMENTS:**

- Your response SHOULD include reasoning text explaining what you're doing
- Your response MUST include AT LEAST ONE bash tool call. You can make MULTIPLE tool calls in a single response when the commands are
independent (e.g., searching multiple files, reading different parts of the codebase).
- Directory or environment variable changes are not persistent. Every action is executed in a new subshell.
```

- However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files

Example of a CORRECT response:
<example_response>
I need to understand the issue. Let me use fastcontext to locate the relevant code first.

[Makes bash tool call: {"command": "cd /app && fastcontext -q 'Find the code related to the issue described' --format concise"}]
</example_response>

Environment Details

- You have a full Linux shell environment
- Always use non-interactive flags (-y, -f) for commands
- Avoid interactive tools like vi, nano, or any that require user input
- You can use bash commands or invoke any tool that is available in the environment
- You can also create new tools or scripts to help you with the task
- If a tool isn't available, you can also install it

Submission

When you've completed your work, you MUST submit your changes as a git patch.
Follow these steps IN ORDER, with SEPARATE commands:

Step 1: Create the patch file
Run `cd /app && git diff -- path/to/file1 path/to/file2 > /app/patch.txt` listing only the source files you modified.
Do NOT commit your changes.

<IMPORTANT>
The patch must only contain changes to the specific source files you modified to fix the issue.
Do not submit file creations or changes to any of the following files:

- test and reproduction files
- helper scripts, tests, or tools that you created
- installation, build, packaging, configuration, or setup scripts unless they are directly part of the issue you were fixing
- binary or compiled files
</IMPORTANT>

Step 2: Verify your patch
Inspect patch.txt to confirm it only contains your intended changes and headers show `--- a/` and `+++ b/` paths.

Step 3: Submit (EXACT command required)
You MUST use this EXACT command to submit:

```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat /app/patch.txt
```

If the command fails (nonzero exit status), it will not submit.

<CRITICAL>
- Creating/viewing the patch and submitting it MUST be separate commands (not combined with &&).
- If you modify patch.txt after verifying, you SHOULD verify again before submitting.
- You CANNOT continue working (reading, editing, testing) in any way on this task after submitting.
</CRITICAL>
</instructions>

E.7 GLM-5.1/Kimi-K2.6 SWE-QA

GLM-5.1/Kimi-K2.6 SWE-QA: System Prompt

You are an expert software engineer. You will be given a technical question about
a code repository. Your task is to carefully explore the repository and provide
a detailed, accurate answer.

FastContext

You have access to `fastcontext`, a fast agent specialized for exploring codebases pre-installed in this environment.
Use this when you need to quickly find files by patterns (eg. "src/components/**/*.tsx"), search code for keywords (eg. "API endpoints"),
or answer questions about the codebase (eg. "how do API endpoints work?").

When NOT to use FastContext tool:
- Simple, single or few-step tasks that can be performed by a single agent (using parallel or sequential tool calls) -- just call the tools directly instead.
- For example:
  - If you want to read a specific file path
  - If you are searching for code within a specific file or set of 2-3 files
  - If you are searching for a specific class definition like "class Foo"

Usage notes:
- Provide clear, detailed prompts so the agent can work autonomously and return exactly the information you need.
- When FastContext is done, it will return a single message back to you: A brief summary and a listing of relevant file paths with line ranges.

Usage:
```bash
fastcontext -q "<your detailed prompts>" --format concise
```

GLM-5.1/Kimi-K2.6 SWE-QA: Instance Template

```
<question>
{{task}}
</question>

<instructions>
# Task Instructions

## Overview

You're a software engineer interacting with a local code repository to answer a technical question.
The current working directory is the repository root. Explore the code to understand the architecture,
implementation details, and relationships between components relevant to the question.

For each response:

1. Include a THOUGHT section explaining your reasoning and what you're trying to accomplish
2. Provide one or more bash tool calls to execute

## Recommended Workflow

1. **Use fastcontext to locate relevant code**: Run `fastcontext -q "<detailed description of what to find>" --format concise`
2. Read the identified files to understand the implementation
3. Explore related files and components as needed
4. Formulate a comprehensive, accurate answer

## Command Execution Rules

1. You issue at least one command
2. The system executes the command(s) in a subshell
3. You see the result(s)
4. You write your next command(s)

Each response should include:
1. **Reasoning text** where you explain your analysis
2. At least one tool call with your command

**CRITICAL REQUIREMENTS:**
- Your response SHOULD include reasoning text
- Your response MUST include AT LEAST ONE bash tool call
- Directory or environment variable changes are not persistent -- prefix commands with `cd /path && ...`

## Submission

When you have a complete, well-supported answer, submit it with a SINGLE combined command:
```bash
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat <<'ANSWER'
<your full answer here>
ANSWER
```

<CRITICAL>
- The submission MUST be a single command -- do NOT split into two separate tool calls.
  Calling `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` alone submits an empty answer.
- You CANNOT continue working after submitting.
- Your answer should be thorough and directly address the question with specific references to the code.
</CRITICAL>
</instructions>
```

F Potential Risks

 is designed as a read-only repository exploration subagent, and by itself does not edit files, execute fixes, or submit patches. Nevertheless, improving repository exploration may indirectly strengthen coding agents, including in settings where automated code modification could introduce bugs, insecure changes, or maintenance burden if used without human review. We therefore view as a component that should be integrated with standard software-engineering safeguards, including patch review, test execution, and repository-specific access controls.

A second risk is over-reliance on retrieved evidence. Because returns compact file-line citations, the main agent may place excessive trust in incomplete or overly narrow context. This can lead to missed edge cases or incorrect fixes when the returned evidence omits tests, call sites, configuration files, or unmodified but semantically relevant code. Our experiments partially address this by keeping the main agent responsible for reading, reproducing, editing, and validating the fix, rather than allowing the explorer to directly modify the repository.

Finally, repository exploration can expose sensitive code or proprietary implementation details

when deployed on private codebases. In such settings, model serving and logging should follow the same privacy and access-control policies as the main coding agent. In our experiments, we use public benchmark repositories and record only the trajectories needed for analysis.

G Artifact Use and Intended Use

We use existing software-engineering benchmarks and public repositories only for research evaluation and analysis. The benchmark artifacts used in this work, including SWE-bench-style issue-resolution tasks and repository-level QA tasks, are intended to support research on automated software engineering agents. Our use is consistent with this purpose: we evaluate repository exploration and coding-agent behavior under controlled benchmark settings, and we do not use the artifacts to provide production software maintenance services or to make decisions about repository owners, contributors, or users.

For artifacts created in this work, including trained exploration models, prompt templates, trajectories, and derived evaluation data, the intended use is research on repository exploration and coding-agent efficiency. These artifacts are released for research and reproducibility purposes. They should be used in accordance with the licenses and access conditions of the underlying datasets and repositories from which they are derived. In particular, derived data produced from research benchmarks should not be repurposed outside research contexts when the original benchmark or repository access conditions restrict such use.

H Personally Identifying and Offensive Content

This work does not collect new human-subject data. The data used in our experiments comes from public software repositories and established software-engineering benchmarks. Such artifacts may contain names, usernames, email addresses, commit metadata, issue comments, or other identifiers that are part of normal open-source development records. We do not attempt to infer private attributes, link identities across datasets, or use such information for profiling. Our training and evaluation procedures operate on repository code, issue descriptions, patches, and model trajectories for software-engineering research purposes.

We also do not intentionally collect offensive content. However, public issue reports, comments, tests, or repository text may contain incidental offensive language or sensitive strings. We minimize exposure by using benchmark-provided tasks and repository snapshots, and by reporting aggregate results rather than reproducing raw user discussions. Released derived artifacts should avoid including unnecessary personally identifying information beyond what is already present in the public benchmark artifacts, and users of the artifacts should follow the privacy and content policies of the original datasets and repositories.

I Use of AI Assistants

We used AI assistants in three limited ways. First, we used AI writing assistance to polish the wording of the paper; all technical claims, experimental analyses, and final content were reviewed and are the responsibility of the authors. Second, we used Sonnet 4.6 to generate supervised fine-tuning trajectories for training the exploration models, as described in Appendix A.1. These model-generated traces were used as training supervision for repository exploration behavior and were filtered before inclusion in the SFT corpus. Third, for SWE-QA, we used GPT-5.4 as the LLM-as-judge evaluator, following the benchmark protocol, which requires an LLM judge to assess answer correctness. We did not use AI assistants to generate benchmark labels for issue-resolution tasks, decide non-QA evaluation outcomes, or replace human review of the reported results.

J SWE-bench Pro Subset

We evaluate SWE-bench Pro on the following fixed 200-instance subset. The full JSONL records are released in `tmp/subset-pro.jsonl`; each row contains the original task fields and the `instance_id` listed below.

1. `instance_qutebrowser__qutebrowser-7f9713b20f623fc40473b7167a082d6db0f0fd40-va0fd88aac89cde702ec1ba84877234da33adce8a`
2. `instance_tutao__tutanota-f3ffe17af6e8ab007e8d461355057ad237846d9d-vbc0d9ba8f0071fbe982809910959a6ff8884dbbf`
3. `instance_protonmail__webclients-2c3559cad02d1090985dba7e8eb5a129144d9811`
4. `instance_gravitational__teleport-b4e7cd3a5e246736d3fe8d6886af55030b232277`
5. `instance_element-hq__element-web-4fec436883b601a3cac2d4a58067e597f737b817-vnan`
6. `instance_flipt-io__flipt-7161f7b876773a911afdd804b281e52681cb7321`
7. `instance_internetarchive__openlibrary-43f9e7e0d56a4f1d487533543c17040a029ac501-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4`
8. `instance_ansible__ansible-5f4e332e3762999d94af27746db29ff1729252c1-v0f01c69f1e2528b935359cfe578530722bca2c59`
9. `instance_qutebrowser__qutebrowser-479aa075ac79dc975e2e949e188a328e95bf78ff-vc2f56a753b62a190ddb23cd330c257b9cf560d12`
10. `instance_internetarchive__openlibrary-d8162c226a9d576f094dc1830c4c1ffd0be2dd17-v76304ecdb3a5954fcf13feb710e8c40fcf24b73c`
11. `instance_gravitational__teleport-1a77b7945a022ab86858029d30ac7ad0d5239d00-vee9b09fb20c43af7e520f57e9239bbcf46b7113d`
12. `instance_flipt-io__flipt-507170da0f7f4da330f6732bffdf11c4df7fc192`
13. `instance_flipt-io__flipt-a0cbc0cb65ae601270bdbe3f5313e2dfd49c80e4`
14. `instance_navidrome__navidrome-8e640bb8580affb7e0ea6225c0bbe240186b6b08`
15. `instance_gravitational__teleport-eefac60a350930e5f295f94a2d55b94c1988c04e-vee9b09fb20c43af7e520f57e9239bbcf46b7113d`
16. `instance_ansible__ansible-12734fa21c08a0ce8c84e533abdc560db2eb1955-v7eee2454f617569fd6889f2211f75bc02a35f9f8`
17. `instance_internetarchive__openlibrary-3f580a5f244c299d936d73d9e327ba873b6401d9-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4`
18. `instance_NodeBB__NodeBB-97c8569a798075c50e93e585ac741ab55cb7c28b-vf2cf3cbd463b7ad942381f1c6d077626485a1e9e`
19. `instance_protonmail__webclients-815695401137dac2975400fc610149a16db8214b`
20. `instance_ansible__ansible-622a493ae03bd5e5cf517d336fc426e9d12208c7-v906c969b551b346ef54a2c0b41e04f632b7b73c2`
21. `instance_flipt-io__flipt-e42da21a07a5ae35835ec54f74004ebd58713874`

22. instance_navidrome__navidrome-de90152a7173039677ac808f5bfb1e644d761336
23. instance_element-hq__element-web-27139ca68eb075a4438c18fca184887002a4ffbc-vnan
24. instance_internetarchive__openlibrary-e010b2a13697de70170033902ba2e27a1e1acbe9-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
25. instance_navidrome__navidrome-3bc9e75b2843f91f6a1e9b604e321c2bd4fd442a
26. instance_future-architect__vuls-4a72295de7b91faa59d90a5bee91535bbe76755d
27. instance_tutao__tutanota-de49d486feef842101506adf040a0f00ded59519-v10a26bfb45a064b93f4fc044a0254925037b88f1
28. instance_internetarchive__openlibrary-2abe28b472ffed563a87cfe83685b161b35263b0-v13642507b4fc1f8d234172bf8129942da2c2ca26
29. instance_protonmail__webclients-d3e513044d299d04e509bf8c0f4e73d812030246
30. instance_internetarchive__openlibrary-a7b7dc5735a1b3a9824376b1b469b556dd413981-va4315b5dc369c1ef66ae22f9ae4267aa3114e1b3
31. instance_internetarchive__openlibrary-4a5d2a7d24c9e4c11d3069220c0685b736d5ecde-v13642507b4fc1f8d234172bf8129942da2c2ca26
32. instance_ansible__ansible-3db08adbb1cc6aa9941be5e0fc810132c6e1fa4b-vba6da65a0f3baefda7a058ebbd0a8dcafb8512f5
33. instance_internetarchive__openlibrary-123e6e5e1c85b9c07d1e98f70bfc480bc8016890-v2733ff199fb72f0d033a30dc62cb0a4742e3a7f4
34. instance_internetarchive__openlibrary-e1e502986a3b003899a8347ac8a7ff7b08cbfc39-v08d8e8889ec945ab821fb156c04c7d2e2810debb
35. instance_element-hq__element-web-776ffa47641c7ec6d142ab4a47691c30ebf83c2e
36. instance_qutebrowser__qutebrowser-44e64199ed38003253f0296badd4a447645067b6-v2ef375ac784985212b1805e1d0431dc8f1b3c171
37. instance_gravitational__teleport-ad41b3c15414b28a6cec8c25424a19bfa7abd0e9-vee9b09fb20c43af7e520f57e9239bbcf46b7113d
38. instance_ansible__ansible-984216f52e76b904e5b0fa0fb956ab4f1e0a7751-v1055803c3a812189a1133297f7f5468579283f86
39. instance_internetarchive__openlibrary-c12943be1db80cf1114bc267ddf4f9933aca9b28-v2c55207218fb8a0138425cbf7d9675272e240b90
40. instance_navidrome__navidrome-5001518260732e36d9a42fb8d4c054b28afab310
41. instance_future-architect__vuls-abd80417728b16c6502067914d27989ee575f0ee
42. instance_future-architect__vuls-6eff6a9329a65cc412e79b8f82444dfa3d0f0b5a
43. instance_ansible__ansible-fb144c44144f8bd3542e71f5db62b6d322c7bd85-vba6da65a0f3baefda7a058ebbd0a8dcafb8512f5
44. instance_internetarchive__openlibrary-3f7db6bbbcc7c418b3db72d157c6aed1d45b2ccf-v430f20c722405e462d9ef44dee7d34c41e76fe7a
45. instance_gravitational__teleport-a95b3ae0667f9e4b2404bf61f51113e6d83f01cd
46. instance_gravitational__teleport-02d1efb8560a1aa1c72cfb1c08edd8b84a9511b4-vce94f93ad1030e3136852817f2423c1b3ac37bc4
47. instance_internetarchive__openlibrary-11838fad1028672eb975c79d8984f03348500173-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
48. instance_gravitational__teleport-629dc432eb191ca479588a8c49205debb83e80e2
49. instance_protonmail__webclients-1917e37f5d9941a3459ce4b0177e201e2d94a622
50. instance_ansible__ansible-6cc97447aac5816745278f3735af128afb255c81-v0f01c69f1e2528b935359cfe578530722bca2c59
51. instance_gravitational__teleport-3a5c1e26394df2cb4fb3f01147fb9979662972c5-vee9b09fb20c43af7e520f57e9239bbcf46b7113d
52. instance_ansible__ansible-e64c6c1ca50d7d26a8e7747d8eb87642e767cd74-v0f01c69f1e2528b935359cfe578530722bca2c59
53. instance_gravitational__teleport-007235446f85b1cbaef92664c3b3867517250f21
54. instance_internetarchive__openlibrary-910b08570210509f3bcfebf35c093a48243fe754-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4

55. instance_navidrome__navidrome-3853c3318f67b41a9e4cb768618315ff77846fdb
56. instance_internetarchive__openlibrary-d109cc7e6e161170391f98f9a6fa1d02534c18e4-ve8c8d62a2b60610a3c4631f5f23ed866bada9818
57. instance_protonmail__webclients-caf10ba9ab2677761c88522d1ba8ad025779c492
58. instance_protonmail__webclients-b9387af4cdf79c2cb2a221dea33d665ef789512e
59. instance_flipt-io__flipt-524f277313606f8cd29b299617d6565c01642e15
60. instance_ansible__ansible-f86c58e2d235d8b96029d102c71ee2dfafd57997-v0f01c69f1e2528b935359cfe578530722bca2c59
61. instance_element-hq__element-web-72a8f8f03b1a01bb70ef8a5bb61759416991b32c-vnan
62. instance_ansible__ansible-d62496fe416623e88b90139dc7917080cb04ce70-v0f01c69f1e2528b935359cfe578530722bca2c59
63. instance_gravitational__teleport-1330415d33a27594c948a36d9d7701f496229e9f
64. instance_internetarchive__openlibrary-1351c59fd43689753de1fca32c78d539a116ffc1-v29f82c9cf21d57b242f8d8b0e541525d259e2d63
65. instance_internetarchive__openlibrary-72321288ea790a3ace9e36f1c05b68c93f7eec43-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
66. instance_gravitational__teleport-32bcd71591c234f0d8b091ec01f1f5cbfdc0f13c-vee9b09fb20c43af7e520f57e9239bbcf46b7113d
67. instance_qutebrowser__qutebrowser-8d05f0282a271bfd45e614238bd1b555c58b3fc1-v35616345bb8052ea303186706cec663146f0f184
68. instance_internetarchive__openlibrary-53e02a22972e9253aeded0e1981e6845e1e521fe-vfa6ff903cb27f336e17654595dd900fa943dcd91
69. instance_tutao__tutanota-befce4b146002b9abc86aa95f4d57581771815ce-vee878bb72091875e912c52fc32bc60ec3760227b
70. instance_protonmail__webclients-cba6ebbd0707caa524ffee51c62b197f6122c902
71. instance_qutebrowser__qutebrowser-6dd402c0d0f7665d32a74c43c5b4cf5dc8aff28d-v5fc38aaf22415ab0b70567368332beee7955b367
72. instance_flipt-io__flipt-29d3f9db40c83434d0e3cc082af8baec64c391a9
73. instance_protonmail__webclients-5f0745dd6993bb1430a951c62a49807c6635cd77
74. instance_future-architect__vuls-d576b6c6c15e56c47cc3e26f5878867677d4a9ea
75. instance_future-architect__vuls-ca3f6b1dbf2cd24d1537bfda43e788443ce03a0c
76. instance_flipt-io__flipt-56a620b8fc9ef7a0819b47709aa541cdfdbba00b
77. instance_qutebrowser__qutebrowser-f7753550f2c1dcb2348e4779fd5287166754827e-v059c6fdc75567943479b23ebca7c07b5e9a7f34c
78. instance_element-hq__element-web-fe14847bb9bb07cab1b9c6c54335ff22ca5e516a-vnan
79. instance_internetarchive__openlibrary-bdba0af0f6cbaca8b5fc3be2a3080f38156d9c92-ve8c8d62a2b60610a3c4631f5f23ed866bada9818
80. instance_tutao__tutanota-f373ac3808deefce8183dad8d16729839cc330c1-v2939aa9f4356f0dc9f523ee5ce19d09e08ab979b
81. instance_ansible__ansible-e22e103cdf8edc56ff7d9b848a58f94f1471a263-v1055803c3a812189a1133297f7f5468579283f86
82. instance_navidrome__navidrome-3982ba725883e71d4e3e618c61d5140eeb8d850a
83. instance_navidrome__navidrome-d0dceae0943b8df16e579c2d9437e11760a0626a
84. instance_element-hq__element-web-6205c70462e0ce2e1e77afb3a70b55d0fdfe1b31-vnan
85. instance_NodeBB__NodeBB-84e065752f6d7fbe5c08cbf50cb173ffb866b8fa-vf2cf3cbd463b7ad942381f1c6d077626485a1e9e
86. instance_qutebrowser__qutebrowser-ef5ba1a0360b39f9eff027fbdc57f363597c3c3b-v363c8a7e5ccdf6968fc7ab84a2053ac78036691d
87. instance_navidrome__navidrome-669c8f4c49a7ef51ac9a53c725097943f67219eb
88. instance_internetarchive__openlibrary-7bf3238533070f2d24bafbb26eedf675d51941f6-v08d8e8889ec945ab821fb156c04c7d2e2810debb
89. instance_ansible__ansible-709484969c8a4ffd74b839a673431a8c5caa6457-vba6da65a0f3baefda7a058ebbd0a8dcafb8512f5

90. instance_internetarchive__openlibrary-5069b09e5f64428dce59b33455c8bb17fe577070-v8717e18970bcdc4e0d2cea3b1527752b21e74866
91. instance_ansible__ansible-489156378c8e97374a75a544c7c9c2c0dd8146d1-v390e508d27db7a51eece36bb6d9698b63a5b638a
92. instance_flipt-io__flipt-86906cbfc3a5d3629a583f98e6301142f5f14bdb-v6bea0cc3a6fc532d7da914314f2944fc1cd04dee
93. instance_qutebrowser__qutebrowser-99029144b5109bb1b2a53964a7c129e009980cd9-va0fd88aac89cde702ec1ba84877234da33adce8a
94. instance_navidrome__navidrome-8383527aaba1ae8fa9765e995a71a86c129ef626
95. instance_internetarchive__openlibrary-08ac40d050a64e1d2646ece4959af0c42bf6b7b5-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
96. instance_flipt-io__flipt-84806a178447e766380cc66b14dee9c6eeb534f4
97. instance_protonmail__webclients-8142704f447df6e108d53cab25451c8a94976b92
98. instance_tutao__tutanota-1ff82aa365763cee2d609c9d19360ad87fdf2ec7-vc4e41fd0029957297843cb9dec4a25c7c756f029
99. instance_flipt-io__flipt-c1728053367c753688f114ec26e703c8fdeda125
100. instance_future-architect__vuls-4c04acbd9ea5b073efe999e33381fa9f399d6f27
101. instance_ansible__ansible-eea46a0d1b99a6dadedbb6a3502d599235fa7ec3-v390e508d27db7a51eece36bb6d9698b63a5b638a
102. instance_flipt-io__flipt-96820c3ad10b0b2305e8877b6b303f7fafdf815f
103. instance_flipt-io__flipt-2ca5dfb3513e4e786d2b037075617cccc286d5c3
104. instance_future-architect__vuls-999529a05b202b0fd29c6fca5039a4c47a3766bb
105. instance_gravitational__teleport-eda668c30d9d3b56d9c69197b120b01013611186
106. instance_future-architect__vuls-ef2be3d6ea4c0a13674aaab08b182eca4e2b9a17-v264a82e2f4818e30f5a25e4da53b27ba119f62b5
107. instance_qutebrowser__qutebrowser-cf06f4e3708f886032d4d2a30108c2fddb042d81-v2ef375ac784985212b1805e1d0431dc8f1b3c171
108. instance_ansible__ansible-cb94c0cc550df9e98f1247bc71d8c2b861c75049-v1055803c3a812189a1133297f7f5468579283f86
109. instance_flipt-io__flipt-abaa5953795afb9c621605bb18cb32ac48b4508c
110. instance_gravitational__teleport-47530e1fd8bfb84ec096ebcbbc29990f30829655-vee9b09fb20c43af7e520f57e9239bbcf46b7113d
111. instance_protonmail__webclients-2f2f6c311c6128fe86976950d3c0c2db07b03921
112. instance_internetarchive__openlibrary-d40ec88713dc95ea791b252f92d2f7b75e107440-v13642507b4fc1f8d234172bf8129942da2c2ca26
113. instance_internetarchive__openlibrary-bb152d23c004f3d68986877143bb0f83531fe401-ve8c8d62a2b60610a3c4631f5f23ed866bada9818
114. instance_flipt-io__flipt-6fd0f9e2587f14ac1fdd1c229f0bcae0468c8daa
115. instance_qutebrowser__qutebrowser-5e0d6dc1483cb3336ea0e3dcbd4fe4aa00fc1742-v5149fcda2a9a6fe1d35dfed1bade1444a11ef271
116. instance_navidrome__navidrome-3972616585e82305eaf26aa25697b3f5f3082288
117. instance_protonmail__webclients-d494a66038112b239a381f49b3914caf8d2ef3b4
118. instance_future-architect__vuls-139f3a81b66c47e6d8f70ce6c4afe7a9196a6ea8
119. instance_tutao__tutanota-8513a9e8114a8b42e64f4348335e0f23efa054c4-vee878bb72091875e912c52fc32bc60ec3760227b
120. instance_protonmail__webclients-09fcf0dbdb87fa4f4a27700800ee4a3caed8b413
121. instance_gravitational__teleport-3ff75e29fb2153a2637fe7f83e49dc04b1c99c9f
122. instance_internetarchive__openlibrary-ba3abfb6af6e722185d3715929ab0f3e5a134eed-v76304ecdb3a5954fcf13feb710e8c40fcf24b73c
123. instance_flipt-io__flipt-0b119520afca1cf25c470ff4288c464d4510b944
124. instance_ansible__ansible-1ee70fc272aff6bf3415357c6e13c5de5b928d9b-v1055803c3a812189a1133297f7f5468579283f86
125. instance_navidrome__navidrome-b3980532237e57ab15b2b93c49d5cd5b2d050013

126. instance_qutebrowser__qutebrowser-473a15f7908f2bb6d670b0e908ab34a28d8cf7e2-v363c8a7e5ccdf6968fc7ab84a2053ac78036691d
127. instance_element-hq__element-web-f63160f38459fb552d00fcc60d4064977a9095a6-vnan
128. instance_flipt-io__flipt-8bd3604dc54b681f1f0f7dd52cbc70b3024184b6
129. instance_future-architect__vuls-c11ba27509f733d7d280bdf661cbbe2e7a99df4c
130. instance_navidrome__navidrome-6b3b4d83ffcf273b01985709c8bc5df12bbb8286
131. instance_tutao__tutanota-fb32e5f9d9fc152a00144d56dd0af01760a2d4dc-vc4e41fd0029957297843cb9dec4a25c7c756f029
132. instance_element-hq__element-web-1216285ed2e82e62f8780b6702aa0f9abdda0b34-vnan
133. instance_ansible__ansible-395e5e20fab9cad517243372fa3c3c5d9e09ab2a-v7eee2454f617569fd6889f2211f75bc02a35f9f8
134. instance_gravitational__teleport-769b4b5eec7286b7b14e179f2cc52e6b15d2d9f3-v626ec2a48416b10a88641359a169d99e935ff037
135. instance_ansible__ansible-34db57a47f875d11c4068567b9ec7ace174ec4cf-v1055803c3a812189a1133297f7f5468579283f86
136. instance_NodeBB__NodeBB-f1a80d48cc45877fcbadf34c2345dd9709722c7f-v4fbcfae8b15e4ce5d132c408bca69ebb9cf146ed
137. instance_element-hq__element-web-aeabf3b18896ac1eb7ae9757e66ce886120f8309-vnan
138. instance_element-hq__element-web-494d9de6f0a94ffb491e74744d2735bce02dc0ab-vnan
139. instance_gravitational__teleport-2bb3bbbd8aff1164a2353381cb79e1dc93b90d28-vee9b09fb20c43af7e520f57e9239bbcf46b7113d
140. instance_element-hq__element-web-56c7fc1948923b4b3f3507799e725ac16bcf8018-vnan
141. instance_navidrome__navidrome-28389fb05e1523564dfc61fa43ed8eb8a10f938c
142. instance_element-hq__element-web-b007ea81b2ccd001b00f332bee65070aa7fc00f9-vnan
143. instance_qutebrowser__qutebrowser-c580ebf0801e5a3ecabc54f327498bb753c6d5f2-v2ef375ac784985212b1805e1d0431dc8f1b3c171
144. instance_future-architect__vuls-bff6b7552370b55ff76d474860eead4ab5de785a-v1151a6325649aaf997cd541ebe533b53fddf1b07
145. instance_qutebrowser__qutebrowser-2dd8966fdcf11972062c540b7a787e4d0de8d372-v363c8a7e5ccdf6968fc7ab84a2053ac78036691d
146. instance_ansible__ansible-a02e22e902a69aeb465f16bf03f7f5a91b2cb828-vba6da65a0f3baefda7a058ebbd0a8dcafb8512f5
147. instance_qutebrowser__qutebrowser-1a9e74bfaf9a9db2a510dc14572d33ded6040a57-v2ef375ac784985212b1805e1d0431dc8f1b3c171
148. instance_NodeBB__NodeBB-22368b996ee0e5f11a5189b400b33af3cc8d925a-v4fbcfae8b15e4ce5d132c408bca69ebb9cf146ed
149. instance_internetarchive__openlibrary-308a35d6999427c02b1dbf5211c033ad3b352556-ve8c8d62a2b60610a3c4631f5f23ed866bada9818
150. instance_qutebrowser__qutebrowser-66cfa15c372fa9e613ea5a82d3b03e4609399fb6-v363c8a7e5ccdf6968fc7ab84a2053ac78036691d
151. instance_internetarchive__openlibrary-0dc5b20fa186f9714f8a838178597e69f549d026-v2d9a6c849c60ed19fd0858ce9e40b7cc8e097e59
152. instance_protonmail__webclients-281a6b3f190f323ec2c0630999354fafb84b2880
153. instance_tutao__tutanota-fe240cbf7f0fdd6744ef7bef8cb61676bcdbb621-vc4e41fd0029957297843cb9dec4a25c7c756f029
154. instance_element-hq__element-web-ca8b1b04effb4fec0e1dd3de8e3198eeb364d50e-vnan
155. instance_ansible__ansible-d33bedc48fdd933b5abd65a77c081876298e2f07-v0f01c69f1e2528b935359cfe578530722bca2c59
156. instance_navidrome__navidrome-0488fb92cb02a82924fb1181bf1642f2e87096db
157. instance_NodeBB__NodeBB-04998908ba6721d64eba79ae3b65a351dcfbc5b5-vnan
158. instance_NodeBB__NodeBB-3c85b944e30a0ba8b3ec9e1f441c74f383625a15-v4fbcfae8b15e4ce5d132c408bca69ebb9cf146ed
159. instance_element-hq__element-web-53a9b6447bd7e6110ee4a63e2ec0322c250f08d1-vnan
160. instance_qutebrowser__qutebrowser-fd6790fe8c02b144ab2464f1fc8ab3d02ce3c476-v2ef375ac784985212b1805e1d0431dc8f1b3c171
161. instance_ansible__ansible-106909db8b730480615f4a33de0eb5b710944e78-v0f01c69f1e2528b935359cfe578530722bca2c59
162. instance_navidrome__navidrome-6c6223f2f9db2c8c253e0d40a192e3519c9037d1
163. instance_flipt-io__flipt-1dceb5edf3fa8f39495b939ef9cc0c3dd38fa17d

164. instance_navidrome__navidrome-0130c6dc13438b48cf0fdfab08a89e357b5517c9
165. instance_flipt-io__flipt-b3cd920bbb25e01fdb2dab66a5a913363bc62f6c
166. instance_flipt-io__flipt-ebb3f84c74d61eee4d8c6875140b990eee62e146
167. instance_qutebrowser__qutebrowser-322834d0e6bf17e5661145c9f085b41215c280e8-v488d33dd1b2540b234cbb0468af6b6614941ce8f
168. instance_navidrome__navidrome-97434c1789a6444b30aae5ff5aa124a96a88f504
169. instance_internetarchive__openlibrary-6afdb09df692223c3a31df65cfa92f15e5614c01-v08d8e8889ec945ab821fb156c04c7d2e2810debb
170. instance_future-architect__vuls-0ec945d0510cdebf92cdd8999f94610772689f14
171. instance_protonmail__webclients-b530a3db50cb33e5064464addbcbef1465856ce6
172. instance_protonmail__webclients-da91f084c0f532d9cc8ca385a701274d598057b8
173. instance_future-architect__vuls-8659668177f1feb65963db7a967347a79c5f9c40
174. instance_NodeBB__NodeBB-b1f9ad5534bb3a44dab5364f659876a4b7fe34c1-vnan
175. instance_navidrome__navidrome-bf2bcb12799b21069f137749e0c331f761d1f693
176. instance_NodeBB__NodeBB-f083cd559d69c16481376868c8da65172729c0ca-vnan
177. instance_ansible__ansible-7e1a347695c7987ae56ef1b6919156d9254010ad-v390e508d27db7a51eece36bb6d9698b63a5b638a
178. instance_flipt-io__flipt-f743945d599b178293e89e784b3b2374b1026430
179. instance_internetarchive__openlibrary-2fe532a33635aab7a9bfea5d977f6a72b280a30c-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
180. instance_NodeBB__NodeBB-6489e9fd9ed16ea743cc5627f4d86c72fbdb3a8a-v2c59007b1005cd5cd14cbb523ca5229db1fd2dd8
181. instance_protonmail__webclients-dfe5604193d63bfcb91ce60d62db2f805c43bf11
182. instance_NodeBB__NodeBB-7b8bffd763e2155cf88f3ebc258fa68ebe18188d-vf2cf3cbd463b7ad942381f1c6d077626485a1e9e
183. instance_ansible__ansible-be2c376ab87e3e872ca21697508f12c6909cf85a-vba6da65a0f3baefda7a058ebbd0a8dcafb8512f5
184. instance_future-architect__vuls-4b680b996061044e93ef5977a081661665d3360a
185. instance_element-hq__element-web-7c63d52500e145d6fff6de41dd717f61ab88d02f-vnan
186. instance_element-hq__element-web-e15ef9f3de36df7f318c083e485f44e1de8aad17
187. instance_internetarchive__openlibrary-fad4a40acf5ff5f06cd7441a5c7baf41a7d81fe4-vfa6ff903cb27f336e17654595dd900fa943dcd91
188. instance_ansible__ansible-40ade1f84b8bb10a63576b0ac320c13f57c87d34-v6382ea168a93d80a64aab1fbd8c4f02dc5ada5bf
189. instance_internetarchive__openlibrary-0a90f9f0256e4f933523e9842799e39f95ae29ce-v76304ecdb3a5954fcf13feb710e8c40fcf24b73c
190. instance_qutebrowser__qutebrowser-3fd8e12949b8feda401930574facf09dd4180bba
191. instance_flipt-io__flipt-5af0757e96dec4962a076376d1bedc79de0d4249
192. instance_internetarchive__openlibrary-8a5a63af6e0be406aa6c8c9b6d5f28b2f1b6af5a-v0f5aece3601a5b4419f7ccec1dbda2071be28ee4
193. instance_qutebrowser__qutebrowser-0833b5f6f140d04200ec91605f88704dd18e2970-v059c6fdc75567943479b23ebca7c07b5e9a7f34c
194. instance_future-architect__vuls-e4728e388120b311c4ed469e4f942e0347a2689b-v264a82e2f4818e30f5a25e4da53b27ba119f62b5
195. instance_qutebrowser__qutebrowser-f631cd4422744160d9dcf7a0455da532ce973315-v35616345bb8052ea303186706cec663146f0f184
196. instance_NodeBB__NodeBB-f48ed3658aab7be0f1165d4c1f89af48d7865189-v0495b863a912fbff5749c67e860612b91825407c
197. instance_navidrome__navidrome-8d56ec898e776e7e53e352cb9b25677975787ffc
198. instance_flipt-io__flipt-40007b9d97e3862bcef8c20ae6c87b22ea0627f0
199. instance_element-hq__element-web-75c2c1a572fa45d1ea1d1a96e9e36e303332ecaa-vnan
200. instance_qutebrowser__qutebrowser-9b71c1ea67a9e7eb70dd83214d881c2031db6541-v363c8a7e5ccdf6968fc7ab84a2053ac78036691d

Scaffold	LLM	File-level			Module-level			Function-level		
		F1	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.
RepoSearcher	Qwen3-4B	55.83	59.32	54.48	38.66	35.39	53.53	20.76	15.26	46.71
	Qwen3-30B	67.14	70.94	65.61	25.18	22.24	35.93	15.40	11.82	33.11
LocAgent	Qwen3-4B	61.78	65.13	60.42	43.88	44.80	47.27	28.04	24.66	41.73
	Qwen3-30B	43.14	45.86	42.19	30.09	30.90	32.58	20.63	17.60	30.35
Agentless	Qwen3-4B	55.44	58.72	54.22	42.06	40.15	52.77	23.55	20.70	35.76
	Qwen3-30B	65.89	69.54	64.48	47.29	43.07	65.12	26.38	19.73	51.77
OrcaLoca	Qwen3-4B	64.12	67.70	62.69	49.72	51.34	51.16	41.56	42.61	44.17
	Qwen3-30B	55.93	60.00	54.67	34.30	37.21	33.67	29.07	30.23	28.68
CoSIL	Qwen3-4B	59.47	62.73	58.22	41.15	37.86	56.30	24.21	18.37	52.13
	Qwen3-30B	68.08	71.74	66.63	37.48	33.47	51.84	23.63	18.28	48.18
OpenHands-Bash	Qwen3-4B	49.73	49.69	53.34	19.32	19.86	20.15	13.27	14.17	13.74
	CodeScout-4B	68.52	71.53	67.74	45.97	49.70	44.97	36.78	40.71	35.72
	CodeScout-14B	68.57	71.00	68.69	50.88	53.71	50.88	40.32	**43.74**	40.27
	GPT-5.4	72.34	76.55	70.69	55.16	51.63	71.76	35.91	29.15	70.81
	GLM-5.1	73.88	77.96	72.29	59.31	56.28	73.51	43.50	37.46	72.02
	Kimi-K2.6	71.34	75.15	69.86	59.34	56.85	70.80	43.87	38.68	68.22
	Qwen3-4B	62.57	66.13	61.19	51.25	53.05	53.79	37.80	37.37	47.22
	Qwen3-30B	65.29	69.14	63.78	57.04	56.48	64.04	**42.90**	39.98	60.62
	FC-30B-SFT	**73.71**	**77.76**	**72.13**	**60.35**	**58.43**	**71.17**	40.74	35.32	67.97
	FC-4B-SFT	70.55	74.75	68.85	55.26	53.00	69.25	37.48	32.83	66.82
	FC-4B-RL	71.48	75.35	69.92	56.26	53.80	70.49	38.45	32.79	**68.05**

TABLE 2 Standalone exploration quality on **SWE-bench Verified** using patch-derived reference locations. We report F1, precision, and recall at file, module, and function granularity. **Bold** marks the best result per column excluding GPT-5.4, GLM-5.1, and Kimi-K2.6 rows; underline marks the second best.

Component	Tokens	API cost
Direct main	457k / task	$282.47
Main + 4B-RL	338k / task	$208.92
4B-RL explorer	22.58M total	$4.52
Augmented total	–	$213.44
Net saving	–	$69.03

TABLE 3 Token and cost audit for the GPT-5.4 SWE-bench Multilingual run. Main-agent token averages are the normalized values reported in Table 1; main-agent costs are estimated from these token totals using GPT-5.4-high reasoning prices. The subagent estimate uses the measured 4B-RL token total and the Fireworks 4B–16B serverless tier of $0.20 per 1M tokens, ignoring any caching discount.

Stage	Protocol detail
Repository state	Evaluate on the pre-PR checkout for each SWE-bench Verified instance.
Input	Provide the issue/query and repository workspace to the explorer; the reference patch is hidden.
References	Derive edited files and old-side edited ranges from the reference patch, then map edited ranges to enclosing module/class and function/method spans.
Predictions	Parse `<final_answer>` citations, normalize paths, and map cited line ranges to the same file/module/function target space.
Scoring	Compute per-instance precision, recall, and F1 for each granularity, then report instance-wise averages.

TABLE 4 Standalone exploration evaluation protocol. We follow the CodeScout benchmark setting for patch-derived file/module/function targets, while adapting predictions from 's file-line citation format.